SALA SENIOR · NOVIEMBRE 2026

Entrevista senior de React Native

Una hora de preguntas, con la respuesta escrita para decirla en voz alta, y treinta minutos de live coding.

React Native 0.88 · React 19.3 · Expo SDK 58 · Hermes V1 · Reanimated 4. New Architecture obligatoria desde 0.82.

Responde en voz alta antes de leer la respuesta larga. La versión de treinta segundos es el arranque. El resto es lo que dices si el entrevistador no te suelta.

CÓMO USAR ESTE ARCHIVO

El orden de la hora

No arranques con los años de experiencia. Arranca con el runtime que sabes operar.

"El stack en el que trabajo hoy es React Native 0.88 sobre Expo SDK 58, React 19.3, New Architecture obligatoria y Hermes V1. He sido responsable de listas grandes, de la frontera nativa y del pipeline de EAS. Cuando me preguntan cómo decido, mido el hilo que se quedó corto y después opino."

Si tus años o tu dominio son otros, cambia solo esa frase. No cambies las versiones por unas más viejas para sonar conservador: en noviembre de 2026, 0.87 y 0.88 son las que tienen soporte activo.

A partir de aquí, cada pregunta se responde en voz alta antes de abrir la respuesta. El reloj de la pregunta se pone en cobre cuando te pasas del minuto asignado.

Lo que tú preguntas, si queda un minuto

- ¿En qué versión están de verdad, y queda algún módulo que no haya migrado a New Architecture?
- ¿El OTA firma los updates, y el runtimeVersion es fingerprint o un string que alguien tiene que acordarse de subir?
- ¿Cuál es la pantalla que pierde frames, y ese número se mide en CI o a ojo en el simulador?
- ¿android e ios se generan en prebuild o están commiteados? ¿El día a día es un dev build o todavía intentan caber en Expo Go?

CONTENIDO

Quince preguntas y tres ejercicios

01 · Arquitectura y runtime	**12 min**
New Architecture de punta a punta	5 min
Qué hilo bloqueaste	4 min
Hermes V1, bytecode y OTA	3 min
02 · Listas, renders y jank	**12 min**
FlatList contra FlashList v2	5 min
La lista entera se vuelve a pintar	4 min
UI a 60, JS a 20	3 min
03 · Estado, red y navegación	**10 min**
Estado de una fintech offline	6 min
Stacks, deep links y pantallas congeladas	4 min
04 · Capa nativa	**12 min**
La spec de un TurboModule	5 min
Expo Modules, Nitro, o bare	4 min
React Native dentro de una app nativa	3 min

05 · Gestos, teclado e insets	**8 min**
Worklets y el snap de un sheet	5 min
Teclado, edge-to-edge e insets	3 min
06 · Tests, release y seguridad	**6 min**
Qué testea un senior	3 min
OTA, secretos y crashes	3 min
Live coding	**30 min**
El feed que se repinta entero	12 min
Paginación que no pisa una carrera	10 min
El sheet que persigue al dedo	8 min

Ficha de diez minutos

BLOQUE 01 · 12 MIN

Arquitectura y runtime

New Architecture, hilos y Hermes V1. Aquí se decide si hablas de 2026 o de un blog de 2023.

PREGUNTA 01 · MINUTO 0 A 5

New Architecture de punta a punta

5 min · Si distingue responsabilidades o si dice 'el bridge nuevo' para todo. Si sabe que en las versiones soportadas ya no hay opt-out.

LO QUE TE PREGUNTAN

Explícame la New Architecture de React Native. Quiero JSI, Fabric, TurboModules, Codegen y bridgeless, y el problema concreto que cada pieza le resuelve al bridge.

VERSIÓN DE TREINTA SEGUNDOS

Desde React Native 0.82 la New Architecture no se puede apagar. JSI es la llamada C++ síncrona que elimina la cola JSON. Fabric es el renderer con el shadow tree en C++. TurboModules son módulos perezosos y tipados. Codegen genera el contrato nativo desde la spec de TypeScript. Bridgeless significa que esa cola ya no existe.

RESPUESTA PARA DECIR EN VOZ ALTA

La New Architecture ya no es una migración opcional. En React Native 0.82, octubre de 2025, `newArchEnabled=false` y `RCT_NEW_ARCH_ENABLED=0` se ignoran. En 0.84 el legacy se compiló fuera del binario de iOS y llegaron los binarios precompilados del core. En 0.88, que es el estable de octubre de 2026 junto con Expo SDK 58 y React 19.3, bridgeless ni siquiera es un flag: se eliminaron `bridgelessEnabled` y los inicializadores que lo recibían. Si alguien propone apagarla para estabilizar, en una versión soportada no hay a dónde volver.

El bridge viejo hacía tres cosas mal a la vez. Serializaba cada llamada a JSON, la metía en una cola asíncrona y la deserializaba del otro lado. JS y nativo no compartían memoria. De ahí salían los síntomas que todavía arrastramos en la cabeza: no podías leer layout en el mismo frame, el arranque inicializaba todos los módulos nativos, y un payload grande se copiaba dos veces. Una imagen en base64 o un objeto enorme de una lista era, literalmente, un stringify.

JSI no es el renderer. Es la interfaz C++ que le permite a Hermes guardar una referencia a un HostObject y llamar métodos sin esa cola. La llamada puede ser síncrona. El costo de cruzar sigue existiendo: convertir tipos en el borde no es gratis, y un método síncrono bloquea el hilo de JS. JSI es el mecanismo. Fabric y TurboModules son los dos sistemas que lo usan.

Fabric es el renderer que reemplazó a Paper. El shadow tree vive en C++, Yoga calcula el layout ahí, y un commit monta las vistas nativas en el hilo principal. Eso es lo que hace viable el modelo concurrente de React 19: prioridades, Suspense y `startTransition`, porque el renderer ya no está atado a una cola de mensajes que no se puede interrumpir. En 0.88 hay un cambio de runtime que me gusta citar: los eventos se despachan por el `EventTarget` de la web. Hay camino de capture hasta el target y bubble de vuelta. El sistema de plugins del renderer viejo quedó atrás.

TurboModules son los módulos nativos perezosos. No se construyen en el arranque, se construyen en el primer acceso. El contrato no es un `NativeModules.Foo` sin tipos: es una spec de TypeScript, y Codegen genera desde esa spec la clase abstracta C++, el JNI y el protocolo de Objective-C. Un

método que devuelve `Promise` es asíncrono. Un método que devuelve `boolean` o `string` es síncrono y se queda en el hilo de JS hasta que vuelve. Eso lo reservo para una lectura barata, nunca para disco o para Keystore.

Codegen es lo que impide que las dos orillas se desincronicen. El conjunto de tipos es cerrado: string, number, boolean, objetos planos, arrays y Promise, más los alias de `CodegenTypes`. No cruzan funciones arbitrarias, ni `Map`, ni `Set`, ni un grafo con ciclos. Si el dato no cabe, no lo fuerzo por JSON a mano: paso un path de archivo o diseño un HostObject, sabiendo que eso ya es código C++ mío.

Bridgeless es la consecuencia, no una quinta librería. No hay cola JSON de por medio. La capa de interop que en 0.76 todavía dejaba vivir a algunos view managers viejos no es un plan de producto. Una librería que no migró a TurboModules o a Expo Modules deja de compilar.

SI INSISTEN

Qué decir si te piden el camino de una llamada

1. JS tiene un objeto JSI. Llamar un método no encola un mensaje JSON.
2. Si el método es sync, el hilo de Hermes se queda esperando al nativo. El frame de JS no avanza.
3. Si devuelve una Promise, el trabajo nativo puede irse a otra cola y resolver después. El hilo de JS queda libre, pero si la implementación nativa toca UIKit o un View en el hilo equivocado, el jank se muda al main thread.
4. En Fabric, un update de React reconcilia, calcula mutations sobre el shadow tree de C++ y commitea en el main thread. El layout de Yoga no es el shadow thread de Paper. Corre en C++ y el commit es el momento delicado.

Lo que ya no debes describir como vigente

- El bridge asíncrono como el camino por defecto.
- `NativeModules` sin spec como la forma de publicar API.
- Un flag de arquitectura para "volver a Paper si Fabric falla".
- Flipper como la herramienta con la que inspeccionas ese camino. React Native DevTools reemplazó ese flujo, y el remote debugging de Chrome se eliminó en 0.79.

El matiz que te separa de alguien que memorizó el diagrama

JSI no vuelve mágicamente barata la frontera. Un bucle de JS que llama a un método nativo por cada ítem sigue siendo un error de diseño. Agrupa, o mueve el bucle al lado nativo. Y un TurboModule sync es una forma muy fácil de volver a bloquear el hilo que acababas de liberar.

FOLLOW-UP

¿Qué se rompe en una librería que solo tiene un view manager de Paper?

No monta. En las versiones soportadas el renderer viejo no está. No hay un flag honesto para encenderlo. La salida es un Fabric component, o reescribir el módulo con la API de Expo Modules, que ya genera la integración de New Architecture. Si la librería está abandonada, la decisión de senior es reemplazarla, no pinnear RN 0.76 para siempre. Pinnear una versión fuera de soporte te deja sin parches de seguridad y sin el renderer donde está el trabajo nuevo de React 19.

FOLLOW-UP

¿Suspense en React Native es el mismo que en web?

El modelo de React sí: un componente puede suspender, el boundary muestra el fallback, y con `startTransition` una actualización no urgente no borra la UI que ya tienes. Lo que no es igual es el dato. No hay streaming de RSC en una app nativa típica. Suspendes sobre una promesa que vosotros controláis, por ejemplo la caché de TanStack Query o una lectura de recursos. Fabric es lo que hace que esas prioridades tengan un renderer capaz de respetarlas. Prometer 'Suspense va a arreglar el spinner de la home' sin decir qué promesa suspende es una respuesta de blog.

FRASES QUE TE BAJAN DE SENIOR

- Llamar 'bridge' a JSI, a Fabric y a TurboModules como si fueran lo mismo.
- Asegurar que se puede poner newArchEnabled en false 'mientras estabilizamos'.
- Decir que una llamada JSI es gratis.
- Describir el shadow thread de Paper como el modelo de layout actual.

Cierre. Cierro con la versión: hoy hablo de 0.88, React 19.3, Expo SDK 58, New Architecture obligatoria desde 0.82 y bridgeless siempre encendido.

PREGUNTA 02 · MINUTO 5 A 9

Qué hilo bloqueaste

4 min · Si separa JS, main, layout y el runtime de Reanimated, y si cada uno tiene un síntoma y una herramienta.

LO QUE TE PREGUNTAN

Dibújame los hilos que importan en una app de React Native hoy. Qué se congela si bloqueas cada uno y cómo lo demuestras.

VERSIÓN DE TREINTA SEGUNDOS

El hilo de JS corre Hermes y React. El main thread pinta y recibe el dedo. El layout de Yoga corre en C++ y duele en el commit. El runtime de Reanimated es otro JS, montado en el UI thread. Bloquear el primero atrasa la respuesta de JS con la UI todavía a 60. Bloquear el segundo congela el dedo aunque el profiler de JS esté limpio.

RESPUESTA PARA DECIR EN VOZ ALTA

Hay cuatro sitios donde una app se siente mal, y no comparten presupuesto.

El hilo de JS es el runtime de Hermes. Ahí corre React: render, hooks, los event handlers que no son worklets, casi toda la lógica de producto. Si lo bloqueas, el síntoma es asimétrico. La UI nativa puede seguir a 60 o 120 fps y aun así el toque "no hace nada" durante 200 ms, porque el handler vive en JS. Listas que dejan huecos blancos mientras se montan celdas, un `JSON.parse` de un payload enorme, un filtro de 50.000 ítems dentro del render, un TurboModule síncrono que lee disco. Lo mido con el FPS de JS del monitor de performance y con el sampling profiler de Hermes, que te da stacks de tu propio código, no una barra genérica.

El main thread, UI thread, es UIKit o la view de Android. Pinta, compone y entrega el touch. Si lo bloqueas, se congela el dedo aunque Hermes esté ocioso. Las causas típicas son demasiadas vistas

nativas porque no virtualizaste, decodificar una imagen de 4000 px en un avatar de 48 pt, sombras y blurs caros en Android, o un módulo nativo que hace trabajo de UI en el main porque nadie cambió de cola. En Android lo miro en Perfetto, en el track del main. En iOS, Instruments: Time Profiler y el FPS de Core Animation.

El layout ya no es "el shadow thread" como respuesta completa. En Fabric, Yoga corre en C++ fuera del main y después hay un commit que sí toca el main. Un árbol profundo, mucho texto que medir, o listas anidadas sin virtualizar, se ven como commits largos. El síntoma es un scroll que engancha justo cuando aparecen celdas nuevas, no un bucle infinito de JS.

El cuarto runtime es el de Reanimated. `react-native-worklets` instala otro runtime de JS en el UI thread. Los shared values viven en memoria compartida. Un worklet que corre ahí no pasa por el render de React. Por eso un bottom sheet puede ir a la tasa del dispositivo, 120 Hz en ProMotion, unos 8,3 ms por frame, mientras el hilo de React está ocupado. `scheduleOnRN`, que es el nombre actual de lo que antes era `runOnJS`, cruza de vuelta al runtime de React de forma asíncrona. Llamarlo en cada frame destruye exactamente la ventaja por la que montaste el worklet.

El presupuesto que uso en voz alta: 16,6 ms a 60 Hz y 8,3 ms a 120 Hz. Esos milisegundos son del hilo que pinta. El hilo de JS tiene el mismo reloj de pared, pero perder un frame de JS no es el mismo bug que perder un frame de UI.

SI INSISTEN

Cómo lo digo en una incidencia real

Llego con dos números, no con una sensación. UI fps y JS fps. Si UI está bien y JS no, dejo de mirar sombras y abro el profiler de Hermes. Si JS está bien y UI no, dejo de memoizar componentes y abro Perfetto o Instruments.

`InteractionManager` ya no es la herramienta. En el salto a SDK 58 se eliminó. Lo que queda para ceder el hilo después de una interacción es `requestIdleCallback`. Citar `runAfterInteractions` en 2026 es una forma rápida de demostrar que el código que mantienes no es el que corre en producción.

Colas de módulos

Un TurboModule async no bloquea a JS mientras espera, pero su implementación puede bloquear el main si toca vistas, o bloquear una cola de fondo si hace I/O pesado sin límite. "Es async" no absuelve al código nativo. Pregunto en qué cola corre el trabajo de verdad.

Lo que no mezclo

El hilo de JS de React y el runtime de UI de Reanimated son dos heaps. Un `console.log` dentro de un worklet no es un log del hilo de React. Un `setState` dentro de un worklet no existe: hay que volver con `scheduleOnRN`. Compartir un objeto mutable de JS con un worklet sin que sea shareable es una carrera, no un patrón.

FOLLOW-UP

El scroll va a 60 y aun así la celda tarda en responder al tap. ¿Dónde miras?

Primero el hilo de JS. El tap entra por nativo, pero el `onPress` corre en Hermes. Si en ese momento hay un render caro, un parse o un módulo sync, el handler espera. Confirmo con el profiler de JS alrededor del gesto. Si el profiler de JS está vacío y el tap igual llega tarde, entonces el main thread no está entregando el touch: miro el track de input en Perfetto. No empiezo por `memo` hasta saber cuál de los dos relojes se paró.

FOLLOW-UP

¿Puedo tener dos runtimes de Hermes?

Sí, y de hecho ya los tienes si usas Reanimated: el runtime de React y el runtime de UI. En brownfield la pregunta es otra. Quiero un solo ReactHost para todas las superficies de RN. Dos runtimes de aplicación duplican memoria, módulos y el costo de arranque. El runtime de worklets no cuenta como segundo host de la app: es el mecanismo de animación, no una segunda instancia de tu producto.

FRASES QUE TE BAJAN DE SENIOR

- Un solo 'thread de React Native' para todo.
- Recomendar InteractionManager.runAfterInteractions.
- Creer que useNativeDriver mueve tu lógica de JS al UI thread. Solo mueve una animación de una lista blanca de propiedades.
- Medir jank solo con la sensación, sin separar los dos FPS.

Cierre. Si me dan un vídeo del bug, pido los dos FPS antes de opinar sobre memo o sobre sombras.

PREGUNTA 03 · MINUTO 9 A 12

Hermes V1, bytecode y OTA

3 min · Si entiende bytecode AOT, el crash por versión de bytecode, y que el binario fija el motor.

LO QUE TE PREGUNTAN

¿Por qué Hermes y no JSC o V8? ¿Qué cambió en Hermes V1 y qué implica eso para un update OTA?

VERSIÓN DE TREINTA SEGUNDOS

Hermes compila a bytecode en build time para no parsear en el arranque. V1 es el motor por defecto desde RN 0.84: otro compilador, otra VM y otro formato de bytecode. Un OTA solo puede llevar JS compatible con el Hermes que ya está dentro del binario. Si el bytecode no coincide, la app muere al abrir con wrong bytecode version.

RESPUESTA PARA DECIR EN VOZ ALTA

Hermes es el motor por defecto desde hace años, y desde React Native 0.84, febrero de 2026, el que arranca es Hermes V1. No es un flag que enciendes. Si ya usabas Hermes, el salto a 0.84 te lo puso. JSC dejó de venir dentro de React Native en 0.81. Se puede salir de Hermes, pero es una decisión que te quedas manteniendo tú, con el paquete de comunidad, y Reanimated 4 ni siquiera soporta el motor de V8.

La idea de Hermes no es ganar el benchmark de un bucle que corre una hora. Es arrancar una app móvil. El bundle de release no viaja como texto: `hermesc` lo compila a bytecode, y el dispositivo carga ese bytecode sin parsear ni calentar un JIT al estilo V8. V1 rehízo compilador y VM. El blog oficial habla de más velocidad de ejecución y menos memoria. El formato de bytecode cambió. Entre 0.82 y 0.83 era la versión 96. Entre 0.84 y 0.86 pasó a 98. No me aprendo el número de 0.88 de memoria porque el

contrato importa más que el entero: el bytecode lo tiene que haber emitido el `hermesc` que corresponde al Hermes embebido en ese binario.

Eso es exactamente el crash de OTA que hay que saber contar. La app abre, y en logcat aparece `Wrong bytecode version. Expected 96 but got 98`, a veces en el hilo de recuperación de `expo-updates`. Pasa cuando alguien compiló el update con otro compilador, o forzó Hermes V1 encima de un runtime que todavía espera el bytecode anterior. El dev build puede seguir funcionando porque en desarrollo Metro sirve JS y el dispositivo compila. El release, el que lleva bytecode embebido, muere. Por eso un OTA no es "subir el bundle y ya".

El binario fija el motor. Un update puede cambiar JavaScript. No puede cambiar el motor, ni meter un módulo nativo nuevo, ni un permiso, ni un bytecode de otra versión. `runtimeVersion` —idealmente el fingerprint de Expo, que cambia cuando cambian los inputs nativos— es lo que impide que un update llegue a un binario que no sabe ejecutarlo. Si publicas bytecode, publica también el JS equivalente para binarios que reportan otra versión. Y `expo-updates` tiene recuperación: si el update nuevo crashea al arrancar, puede volver al anterior. Eso se prueba, no se asume.

Dos límites que digo sin que me los pregunten. Primero, el bytecode no es una frontera de seguridad. Se descompila lo bastante bien como para leer tu lógica. Un secreto dentro del bundle es un secreto publicado. Segundo, no asumas el `Intl` de Node o de V8. Fechas, plurales y ordenación se verifican en dispositivo. Y en release quito los `console.log`: en Hermes no son gratis, y Metro ya no los streamea como antes.

SI INSISTEN

Optar por salir de V1

Se puede forzar el `hermes-compiler` viejo con `overrides` y apagar V1 en Gradle y en CocoaPods. En iOS eso además te obliga a dejar los binarios precompilados (`RCT_USE_PREBUILT_RNCORE=0`) y a compilar el core. Es una salida de emergencia de una regresión, no una arquitectura. La que vimos en la serie 0.86 con Reanimated y worklets —más memoria, y en dev un arranque más lento— se corrigió en el propio runtime. Quedarse en el compilador viejo te desalineó el bytecode del resto del equipo.

Qué miro cuando el arranque es lento

Que el release esté cargando `.hbc` y no el bundle de texto. Que no haya un `require` cíclico enorme en el camino del primer render. Que los módulos nativos sean perezosos de verdad y nadie los toque todos en el root. El sampling profiler de Hermes en el arranque dice si el tiempo es JS. Si el tiempo es anterior a JS, es nativo: carga de frameworks, inicialización de SDKs de terceros en el AppDelegate o en `Application`.

Depuración

El depurador remoto de Chrome murió en 0.79. Se usa React Native DevTools. Si alguien del equipo todavía abre `react-native-debugger` contra un runtime nuevo, no va a enganchar.

FOLLOW-UP

¿Subirías el OTA como JS suelto o como bytecode?

Bytecode si el pipeline garantiza que es el de ese Hermes, porque el arranque del update se parece al del binario. JS suelto es más tolerante entre versiones de bytecode, y el motor lo compila en el dispositivo, que es justo el costo que Hermes quería sacar del arranque. En Expo dejo que el pipeline de `expo-updates` haga el emparejamiento y exijo firma de updates. No subo un `.hbc` compilado en el portátil de alguien con otro `hermesc`.

FOLLOW-UP

¿Hermes tiene JIT?

No lo vendo como un V8. El diseño sigue siendo AOT: el trabajo grande de compilar ocurre en el build. V1 mejoró compilador y VM, y hay descripciones de un tier más ligero para funciones calientes, pero en una entrevista me quedo con lo que cambia decisiones: arranque, memoria, formato de bytecode y el hecho de que el pico de un bucle largo no es el objetivo por el que elijo el motor. Si un cálculo numérico largo es el producto, lo bajo a nativo. No cambio el motor de toda la app por eso.

FRASES QUE TE BAJAN DE SENIOR

- Decir que Hermes es más rápido que V8 en cualquier benchmark.
- Creer que un OTA puede actualizar Hermes.
- Guardar API keys en el bundle 'porque va compilado a bytecode'.
- Depurar un release de 0.88 con el remote debugger de Chrome.

Cierre. El motor vive en el binario. El update solo puede traer JS que ese motor sabe ejecutar.

BLOQUE 02 · 12 MIN

Listas, renders y jank

FlashList v2, React Compiler y cómo separar un hilo de JS bloqueado de un main thread bloqueado.

PREGUNTA 04 · MINUTO 12 A 17

FlatList contra FlashList v2

5 min · Si deja de recitar windowSize y habla de reciclaje, identidad de las filas y medición.

LO QUE TE PREGUNTAN

Tenemos un feed de varios miles de filas. Explica cómo virtualiza FlatList, qué cambió FlashList v2, y qué perillas mueven de verdad el FPS.

VERSIÓN DE TREINTA SEGUNDOS

FlatList desmonta lo que sale de la ventana. FlashList v2 recicla celdas sobre la New Architecture y ya no te pide estimatedItemSize. La identidad tiene que ser un id estable. El estado local de una celda reciclada sobrevive al ítem anterior si no lo reseteas. Y una imagen sin tamaño decodifica de más en el main thread.

RESPUESTA PARA DECIR EN VOZ ALTA

FlatList es VirtualizedList. No pinta los miles de filas: mantiene una ventana, mide celdas, y desmonta lo que se aleja. Desmontar es simple de razonar y es también el costo. Cada vez que una fila vuelve a entrar, corre mount, efectos y decodificación otra vez. El estado local se pierde, que a veces es lo que quieres.

Las perillas que sí mueven algo: `initialNumToRender` alcanza para llenar el primer viewport, no para precalentar cien filas. `windowSize` es múltiplo de viewports; subirlo come memoria y tapa huecos, bajarlo hace lo contrario. `maxToRenderPerBatch` y `updateCellsBatchingPeriod` reparten el mount para no clavar un frame. `getItemLayout` se salta la medición asíncrona y es lo que vuelve serio a `scrollToIndex`. `removeClippedSubviews` puede ayudar en Android y recorta mal hijos en absoluto. `keyExtractor` devuelve un id estable. El índice es un bug en cuanto la lista se reordena, se filtra o se le inserta algo arriba.

`extraData` existe para cuando la fila depende de algo que no está dentro del ítem, un modo de selección por ejemplo. Si lo usas como cubo de basura, re-renderizas toda la ventana en cada cambio.

FlashList v2 es otra estrategia. Shopify lo reescribió para la New Architecture. Recicla la instancia, al estilo de RecyclerView, en vez de desmontar. En la v1 tenías que acertar `estimatedItemSize` y, si mentías, tenías huecos y saltos. En la v2 ese estimado ya no es el contrato. Mide y va refinando la posición. `scrollToIndex` calcula, mide vecinos y corrige hasta caer en el ítem. Las listas horizontales pueden tener ítems de alto variable y, metidas dentro de una vertical, coordinan el montaje. En sus números, el área en blanco al hacer scroll baja de forma visible frente a la v1 sobre New Architecture.

El precio del reciclaje es el estado. La misma instancia de `Row` recibe otro `item` sin unmount. Un `useState` de "expandido", un `useEffect` que no depende de `item.id`, o un input a medio escribir, se quedan pegados a la fila de abajo. La corrección es resetear con el id: key interna, o derivar el estado del id en el render. `memo` sigue teniendo sentido para no pintar si el mismo ítem vuelve con las mismas props, pero no evita el reciclaje cuando el ítem sí cambia.

LegendList es la alternativa que miro si el equipo ya la estandarizó o si su modelo de tamaño dinámico encaja mejor. No abro una guerra de librerías. En un proyecto nuevo sobre 0.88, FlashList v2 es la opción por defecto y sé decir por qué: reciclaje, sin estimado obligatorio, y el trabajo de scroll ya no es el de la v1.

Lo que ninguna de las dos perdona: listas virtualizadas anidadas en un ScrollView, que es el warning clásico y además desactiva la ventana. Una horizontal dentro de una vertical se puede, con alto explícito. Un `map` de quinientas filas dentro de un ScrollView, no.

Y las imágenes de la fila. `Image` de React Native decodifica de más. En Expo uso `expo-image` con ancho y alto explícitos, `recyclingKey` igual al id, y caché memory-disk. Un PNG de cámara dentro de un círculo de 36 pt es un problema de main thread, no de FlatList.

SI INSISTEN

Chats y listas invertidas

Para un chat, `inverted` o, mejor cuando aplica, `maintainVisibleContentPosition`, para que un mensaje nuevo no te saque de donde estabas leyendo. El patrón de paginar hacia el pasado mientras llega un mensaje nuevo es exactamente la carrera que después vemos en el live coding: la página vieja no puede reemplazar el array que ya tiene el mensaje optimista.

Cabeceras pegajosas y animaciones de layout

Un sticky header se vuelve caro si su contenido cambia de alto mientras scrolleas. Las layout animations de Reanimated sobre FlashList v2 se pueden, y el propio equipo de Shopify dice que hay que configurarlas con cuidado. No las enciendo en todas las filas "porque queda bien". Una animación de layout es trabajo de commit. En un feed, animo el gesto, no la aparición de cada celda.

Cómo decido en la entrevista, en una frase

Si la fila es barata y son doscientas, FlatList bien escrita basta. Si son miles, alturas variables y el scroll es el producto, FlashList v2. Si además hay estado local en la celda, enseño el reset por id antes de enseñar el import.

FOLLOW-UP

¿Por qué scrollToIndex se queda a mitad o salta?

Porque la lista no conoce el offset hasta medir, y medir es asíncrono. En FlatList, sin `getItemLayout`, `scrollToIndex` falla o hay que usar `onScrollToIndexFailed` para reintentar. En FlashList v1 un `estimatedItemSize` mal puesto te dejaba cerca pero no encima. En v2 el algoritmo refina: calcula con el mapa de layout, mide vecinos y corrige durante el scroll. Si aún falla, casi siempre es que el alto cambia después del primer layout —una imagen que llega, un texto que pasa de una línea a tres— y nadie avisó. Le doy alto estable o dejo que la lista mida antes de pedir el índice.

FOLLOW-UP

¿onEndReached se dispara en bucle. Qué haces?

No pido la página siguiente dentro del render. Guardo un ref con el cursor que ya pedí y no vuelvo a pedirlo hasta que esa request termina en éxito o en error. Si termina en error, el footer tiene un reintentar explícito: un reintento automático dentro de `onEndReached` es el bucle. Y el threshold se cuenta sobre el largo visible, así que una lista corta que no llena la pantalla dispara el callback al montar. Lo trato como un caso, no como un bug del usuario.

FRASES QUE TE BAJAN DE SENIOR

- keyExtractor con el índice en una lista que se reordena.
- Decir que FlashList v2 sigue exigiendo estimatedItemSize.
- Poner un map dentro de ScrollView 'porque FlatList es complicado'.
- Ignorar que la celda reciclada conserva useState.

Cierre. Virtualizar es no montar. Reciclar es montar una vez y cambiarle los datos. El segundo gana en scroll y pierde si el estado de la celda no entiende que el ítem cambió.

PREGUNTA 05 · MINUTO 17 A 21

La lista entera se vuelve a pintar

4 min · Si mide antes de envolver en memo, y si entiende que el compiler no arregla un prop que sí cambió ni una celda reciclada.

LO QUE TE PREGUNTAN

El padre re-renderiza y todas las filas visibles parpadean. Camina el diagnóstico. ¿Dónde entra React Compiler y dónde memo sigue haciendo falta?

VERSIÓN DE TREINTA SEGUNDOS

Primero el profiler de React Native DevTools, no memo a ciegas. React Compiler 1.0 memoiza solo lo que sigue las reglas de React, y se rinde si mutas. Saco el estado que cambia seguido, un reloj o un texto de búsqueda, del componente que posee la lista. memo no ayuda si le paso un onPress nuevo cada vez. Y en FlashList, además, reseteo el estado local cuando cambia el id.

RESPUESTA PARA DECIR EN VOZ ALTA

No empiezo envolviendo filas en `memo`. Abro React Native DevTools, grabo la interacción y miro qué componente renderizó y por qué. La mayoría de las veces el padre tiene un estado que no tiene nada que ver con la fila: un reloj, el draft del buscador, un contador de no leídos en el header. Ese `setState` vuelve a ejecutar el componente que crea la lista. A partir de ahí hay dos costos distintos. Uno, ejecutar la función del padre. Dos, que cada fila reciba props nuevas y pinte de verdad.

React Compiler 1.0 salió estable en octubre de 2025 y aplica a React Native. Memoiza valores y componentes que respetan las reglas: no mutar props ni state, hooks en orden. En Meta bajó tiempos de interacción de forma medible con la memoria neutra. No es un modo turbo. Si el compiler se rinde —y `eslint-plugin-react-hooks` en el preset recomendado te marca esos bailouts— esa función vuelve al

comportamiento de siempre. El plugin de Babel va el primero en la cadena. El de worklets, `react-native-worklets/plugin`, va el último. Si el preset de Expo del SDK 58 ya lo mete, no lo duplico.

El compiler no puede inventar que dos props son iguales cuando no lo son. Si la fila recibe `now={Date.now()}` o `onPress={() ⇒ open(item.id)}` creado en el render del padre, la prop cambió y la fila debe pintar. La cura no es memo. Es dejar de pasar eso. El reloj vive en su propio componente. El handler recibe el id dentro de la fila y llama a una función estable, `useCallback` o, mejor, una función de módulo que navega. El ítem que viene de la caché de TanStack Query mantiene identidad si el JSON no cambió, gracias al structural sharing. Si en el filtro hago `items.map(clone)`, rompo esa identidad y todas las filas pintan, con compiler y sin él.

Context es la otra fuga. Un provider cuyo `value` es un objeto nuevo en cada render despierta a todos los consumidores, aunque el campo que les importa no haya cambiado. Parto el context, o no uso context para estado frecuente. Zustand con selector, `useStore(s ⇒ s.filter)`, suscribe al componente al pedazo. El servidor no se copia a Zustand. La caché de Query es la fuente de verdad de las entidades. Zustand guarda el draft, la pestaña, el sheet abierto.

`memo` en todas partes tiene costo de comparación y esconde el flujo de datos. Lo dejo donde el profiler muestra un hijo caro con props que de verdad se repiten. Con el compiler encendido, mucho `useCallback` manual sobra. Lo que no sobra nunca es el diagnóstico de FlashList: la instancia se recicla, así que un estado local "esta fila está abierta" tiene que morir cuando `item.id` cambia. El compiler no desmonta esa instancia por ti.

Estilos. `StyleSheet.create` te da un objeto estable y legible. Un objeto inline no es automáticamente un pecado con el compiler, que puede memoizarlo si sus entradas no cambian, pero un estilo que cierra sobre estado sí lo es. Y en listas, el estilo que depende de `selected` tiene que depender de un boolean, no de clonar el ítem.

SI INSISTEN

Orden de diagnóstico que recito si me apuran

1. Profiler: ¿quién renderizó?
2. ¿El state que cambió necesita ser dueño de la lista? Si no, lo bajo a un hijo.
3. ¿Las props de la fila son referencialmente iguales cuando el ítem no cambió?
4. ¿Hay un context o un store sin selector por encima?
5. ¿La lista recicla y el bug es estado viejo, no un render de más?
6. Recién ahí, memo puntual o un bailout del compiler que hay que hacer compilable.

React 19, ya que estamos en 19.3

`forwardRef` ya no hace falta: `ref` es una prop. Eso limpia las filas. No cambia nada del modelo de render. Tampoco lo cambia `use`. Sirve para leer una promesa o un context, no para virtualizar.

El bug del buscador

Cada tecla en un input controlado por el mismo componente que renderiza 5.000 filas es el caso de manual. Aunque las filas estén memoizadas, el padre corre, filtra y crea un array nuevo: eso sí es legítimo. Lo que no es legítimo es que el header, el teclado y un gráfico de la misma pantalla estén dentro de ese mismo componente y no estén aislados. El filtro produce `data`. FlashList recibe `data`. El input es otro componente.

FOLLOW-UP

¿Por qué mi React.memo no hizo nada?

Porque la comparación por defecto es superficial y alguna prop es nueva. Las tres de siempre: el `style={{ }}` inline, el `onPress` inline, y el ítem clonado en un `.map`. Miro también `extraData` con identidad nueva. Y miro si quien renderiza no es React sino la propia lista, llamando `renderItem` y pasando el padre entero como prop. `memo` compara props. Si le pasas `parentState`, va a pintar, y hace bien.

FOLLOW-UP

¿El compiler vuelve inútil a Zustand?

No. El compiler evita renders de hijos cuyas props no cambiaron. No evita que un context despierte a quien lo consume, ni reemplaza un selector. Sigo usando Query para el servidor y un store pequeño para UI. Lo que sí vuelve ruido es el `useCallback` ceremonial en cada handler. Lo quito cuando el compiler está encendido y el profiler está limpio, no antes.

FRASES QUE TE BAJAN DE SENIOR

- memo en cada componente del repo como estándar de equipo.
- Copiar la respuesta de React Query dentro de Zustand 'por si acaso'.
- Creer que el compiler acelera un JSON.parse o un sort de 50.000 elementos.
- Un solo Context con user, theme, cart y el draft del buscador.

Cierre. Mido, saco el estado ruidoso del dueño de la lista, y dejo que el compiler cubra el memo aburrido. El estado de una celda reciclada lo trato aparte.

PREGUNTA 06 · MINUTO 21 A 24

UI a 60, JS a 20

3 min · Si tiene un ejemplo concreto de cada hilo y una herramienta, no una lista de librerías.

LO QUE TE PREGUNTAN

La interfaz se ve fluida y aun así la app responde tarde. Otras veces el dedo se pega y el profiler de JS está limpio. ¿Cómo separas los dos janks?

VERSIÓN DE TREINTA SEGUNDOS

Dos contadores: JS fps y UI fps. JS bajo con UI alta es Hermes: parse, sort, render, módulo síncrono. UI baja con JS alta es el main thread: vistas de más, imágenes, sombras, layout. Perfetto o Instruments para el main. Sampling profiler de Hermes para JS. En producción no 'optimizo' sin un número de frames perdidos.

RESPUESTA PARA DECIR EN VOZ ALTA

Los trato como dos incidentes distintos que se sienten parecido.

Caso A: el monitor muestra UI a 60 y JS a 15. El dedo llega, la pantalla no se congela, pero el handler corre tarde y la lista deja huecos. El trabajo está en Hermes. Ejemplos que he visto de verdad: un

`JSON.parse` del payload completo en el camino del render, ordenar y clonar decenas de miles de movimientos cada vez que cambia un filtro, un `console.log` de ese array en release, y un TurboModule síncrono llamado dentro de `renderItem`. También un render de React que produce cientos de nodos porque alguien sustituyó la lista virtualizada por un `map`. Ahí abro el sampling profiler de Hermes y miro el stack propio. Si la función caliente es mía, la parto, la memoizo de verdad —resultado, no componente— o la saco del frame. Si la función caliente es el reconciler, el problema es el tamaño del árbol, no el algoritmo de JS.

Caso B: JS a 60, UI a 20. Hermes está esperando. El main thread no termina el frame. Ejemplos: una pantalla con demasiadas vistas nativas, sombras grandes en Android, un blur, una imagen que se decodifica a resolución de cámara, texto que se vuelve a medir en cada frame porque su ancho depende de algo animado, y una animación de Reanimated que mueve `width`, `height` o `top` en vez de `transform` y `opacity`. Animar layout obliga a Yoga y al commit. Animar transform lo resuelve el compositor. En Android grabo un trace de Perfetto de tres segundos del gesto y miro el main. En iOS, Instruments. No me hace falta un perfil de veinte minutos para ver un frame de 40 ms.

Reanimated tiene su propio capítulo desde que la New Architecture es obligatoria. Hubo regresiones reales de scroll: parpadeo de cabeceras animadas y menos FPS cuando hay muchas vistas animadas. Las mitigaciones documentadas en la serie 4 dependen de la versión: flags como `USE_COMMIT_HOOK_ONLY_FOR_REACT_COMMITS` o `DISABLE_COMMIT_PAUSING_MECHANISM`, y en el lado de RN `preventShadowTreeCommitExhaustion`. No los recito como un conjuro. Digo que existen, que se leen en la guía de la versión que el proyecto tiene, y que bajar a Reanimated 3 no es un plan: la 3 no tiene soporte para este runtime.

En CI no dejo el jank a ojo. Un test de Reassure atrapa renders de más en una interacción. Un benchmark de scroll, aunque sea uno solo sobre la lista que nos importa, atrapa el frame time. En producción quiero crashes de JS y crashes nativos por separado, y un muestreo de frames perdidos, no un log de cada frame.

Y el orden de un arranque lento es el mismo método. Si el tiempo es anterior al primer JS, es nativo: SDKs en el arranque, runtime que se carga en el tap en vez de durante el splash. Si es JS, es el grafo de imports y el primer render. Hermes V1 ayuda al parse. No esconde un `require` de toda la app en el root.

SI INSISTEN

Lo que le enseño a un perfil de Perfetto

Un frame de UI tiene que caber en 16 ms, u 8 ms si la pantalla es de 120 Hz. Busco un slice largo en el main: `inflate`, `draw`, `measure`, decodificación de bitmap. Si el slice largo está en el hilo de Hermes, vuelvo al caso A aunque el síntoma lo haya sentido en el dedo.

requestIdleCallback

Cuando el trabajo es real pero no es del frame —precargar la página siguiente, calcular un resumen—, lo cedo con `requestIdleCallback`. No con `InteractionManager`, que ya no está. Y no lo uso para esconder un sort de 50.000 elementos: eso se hace en otro sitio o se pagina en el servidor.

El falso "va a 60"

El monitor de desarrollo tiene costo. Un número malo en dev no siempre es un número malo en release, sobre todo con Hermes en dev sirviendo JS sin el bytecode final. Confirmo el jank en un release, en un dispositivo de gama media, con el gesto grabado. Optimizar el dev es una forma clásica de perder la tarde.

FOLLOW-UP

¿Qué animación nunca pones en el hilo de JS?

Cualquiera que acompañe al dedo. Un sheet, un swipe para borrar, un header que colapsa con el scroll. Si el valor se actualiza en el handler de JS y después cruza para mover una vista, a 120 Hz llegas tarde en cuanto React pinta otra cosa. Eso vive en un shared value. Lo que sí dejo en JS es la decisión de negocio que ocurre una vez: cerró, borró, navegó. Una llamada a `scheduleOnRN` al soltar, no cuarenta por segundo.

FRASES QUE TE BAJAN DE SENIOR

- Proponer Flipper como primer paso.
- Optimizar en dev y dar el bug por cerrado.
- Animar height con Reanimated en una lista y llamarlo GPU.
- Un solo número de FPS.

Cierre. Primero qué reloj se paró. Después la herramienta de ese reloj. El memo viene tercero, si viene.

BLOQUE 03 · 10 MIN

Estado, red y navegación

Offline-first de verdad y el contrato de deep links, stacks nativos y pantallas congeladas.

PREGUNTA 07 · MINUTO 24 A 30

Estado de una fintech offline

6 min · Si no duplica el servidor en un store global, y si distingue una acción que se puede encolar de un pago.

LO QUE TE PREGUNTAN

Diseña el estado de una app financiera que tiene que abrir sin red, mostrar saldos y encolar algunas acciones. Qué va en Query, qué en Zustand, qué en MMKV, qué en el llavero.

VERSIÓN DE TREINTA SEGUNDOS

TanStack Query es la caché del servidor. Zustand es UI. MMKV persiste la caché y la bandeja de salida. El llavero guarda el token y la clave de cifrado, nunca el saldo en claro dentro de AsyncStorage. Un pago no es optimista: se muestra pendiente hasta el ack. Una categoría sí se puede encolar. Context no lleva estado que cambie con el teclado.

RESPUESTA PARA DECIR EN VOZ ALTA

Parto en cuatro cajones y no dejo que se mezclen.

El servidor vive en TanStack Query v5. La `queryKey` es un array que describe la entidad y sus parámetros, no un string armado a mano. `staleTime` dice cuánto puedo mostrar sin refetch: un saldo puede ser 15 o 30 segundos, un catálogo de bancos puede ser horas. `staleTime: 0` significa siempre vencido, que es el default, y por eso la app refetcha en cada foco aunque el dato no haya podido cambiar. `gcTime` es cuánto sobrevive en memoria una query que nadie observa. El `focusManager` se engancha a `AppState`, no a un evento de ventana que en nativo no existe. El `onlineManager` se engancha a NetInfo. Sin eso, Query cree que siempre hay red y que la app nunca volvió a primer plano.

La persistencia de esa caché pasa por `persistQueryClient` con un persister sobre MMKV. MMKV, en las versiones actuales, lee y escribe en síncrono por JSI. AsyncStorage es asíncrono y pierde el arranque: hidratas tarde y el saldo parpadea vacío. La hidratación tiene una compuerta. No pinto la home hasta saber si hay caché, o pinto un esqueleto honesto. Lo que no persisto en claro es cualquier cosa que, junto con el dispositivo desbloqueado, ya sea sensible de más. MMKV se puede cifrar. La clave de ese cifrado vive en `expo-secure-store`, que en iOS es Keychain y en Android es Keystore. El access token y el refresh token viven ahí, no en la query del usuario, no en Zustand, no en un header hardcodeado.

Zustand guarda lo que no es servidor: pestaña, draft de un formulario, sheet abierto, el id seleccionado. Selectores para no suscribir la pantalla entera. No copio el saldo a Zustand "para tenerlo a mano". En cuanto hay dos fuentes de verdad, el bug es de caché y tarda una semana en verse.

Context queda para dependencias estables: el tema, si no cabe en el sistema de estilos, y los providers de librería. Un context con el objeto de sesión nuevo en cada refresh re-renderiza el árbol. El carrito de una fintech no es un context.

Mutaciones. `useMutation` con `onMutate` que deja snapshot, `onError` que revierte, `onSettled` que invalida. Eso sirve para categorizar un movimiento o marcar un beneficiario. No sirve para mover

dinero. Un pago se muestra como pendiente con un id de idempotencia que el cliente manda y el servidor respeta. Si reintento, no cobro dos veces. Si no hay red, el pago no entra en la bandeja: la UI dice que no hay red y no simula un comprobante. La bandeja de salida, en MMKV, es para lo que sí es seguro reintentar. Al volver la red se drena con backoff. El conflicto, cuando existe, lo gana el servidor con un `etag` o una versión, y la UI muestra el choque en los campos raros. No monto un CRDT para un saldo.

El orden de un arranque sin red queda así. SecureStore me dice si hay sesión. MMKV hidrata la caché cifrada. La compuerta se abre con datos viejos y un sello de "actualizado a las". Cuando hay red, Query refetcha lo vencido. Si el refetch falla, dejo lo viejo y un banner. No vacío la lista para enseñar un error.

Y una regla de producto que digo en la entrevista porque es de senior y no de librería: el número que el usuario va a usar para tomar una decisión de dinero lleva frescura visible. Un saldo de hace tres días no se viste de saldo de ahora.

SI INSISTEN

Números que conviene tener en la boca

- `staleTime: Infinity` y la invalidación manual cuando el dominio es "esto solo cambia si yo lo cambio".
- `gcTime` no es `staleTime`. Una query puede estar vencida y seguir en memoria.
- `structuralSharing` viene activo. Si el servidor devuelve lo mismo, la referencia del `data` se conserva y la fila memoizada no pinta. Un `select` que siempre clona anula esa ventaja.
- `retry` en una mutación de pago es peligroso si no hay llave de idempotencia. En una query de lectura, reintentar sí.

Lo que no entra en la bandeja

Cualquier POST que no sea idempotente, cualquier cosa que el usuario cree que ya ocurrió porque vio un check, y cualquier payload con el PAN o el secreto de una tarjeta. La bandeja es un diario de intenciones aburridas, no una segunda cola de pagos.

Por qué no Redux, dicho sin pelear

Redux Toolkit sigue siendo válido si el equipo ya tiene middleware, auditoría y gente que lo mantiene. No lo introduzco en una app nueva para leer un saldo. Query más un store chico cubre el caso. Si me preguntan por acciones globales de logout, una función `reset()` que limpia Query, Zustand y la bandeja, y borra el token del llavero, es más clara que un reducer raíz.

FOLLOW-UP

El usuario mata la app a mitad de un pago pendiente. ¿Qué ve al volver?

El pago no dependía de memoria. El id de idempotencia y el estado 'pendiente' están en MMKV, o mejor, los recupero del servidor, que es la fuente de verdad. Al abrir, una query del pago pregunta. Si el servidor ya lo aplicó, muestro el comprobante. Si sigue pendiente, muestro pendiente. Si no existe, no invento un éxito local. Nunca reconstruyo un 'pagado' solo porque `onMutate` había corrido.

FOLLOW-UP

¿Por qué no persistes toda la caché de Query sin mirar?

Porque la caché tiene basura de sesión: un listado de beneficiarios, un borrador de transferencia que un `select` dejó ahí, a veces tokens si alguien metió mal la query. Persisto un subconjunto, cifrado, con una versión de esquema. Si la versión no coincide al actualizar la app, tiro esa caché y refetcho. Migrar JSON de MMKV a mano en cada release es como no tener persister.

FRASES QUE TE BAJAN DE SENIOR

- Un Context llamado AppState con la sesión, el saldo y el texto del buscador.
- AsyncStorage para el token.
- Pago optimista que muestra comprobante antes del ack.
- Copiar data de useQuery a un useEffect que hace setSaldo.

Cierre. Servidor en Query, UI en un store chico, secreto en el llavero, disco cifrado para la caché, y los pagos no se imaginan.

PREGUNTA 08 · MINUTO 30 A 34

Stacks, deep links y pantallas congeladas

4 min · Si conoce el doble manejo del link, el certificado de Play App Signing, y la diferencia entre congelar y desmontar.

LO QUE TE PREGUNTAN

¿Native stack o stack de JS? ¿Cómo dejas bien hechos los Universal Links y los App Links, y qué ganas congelando pantallas inactivas?

VERSIÓN DE TREINTA SEGUNDOS

El stack nativo, UINavigationController y el back stack de Android, es el default. El stack de JS solo si la transición no cabe en el nativo. Expo Router mapea archivos a rutas. Un link se maneja una vez, no en el initialURL y otra vez en el evento. freezeOnBlur deja la pantalla montada pero sin render. No es un unmount. Y el login no puede volver con el botón atrás.

RESPUESTA PARA DECIR EN VOZ ALTA

El stack por defecto es el nativo, el de `react-native-screens`. En iOS es un `UINavigationController` de verdad, con el gesto de volver del sistema. En Android participa del back stack de verdad. Las transiciones las pinta el sistema, no el hilo de JS. El stack de JS, el que anima con Reanimated, lo dejo para una transición que el contenedor nativo no sabe expresar. Cuesta más, pelea con el teclado y es más fácil de dejar a medias. En una app de producto no es el sitio donde quiero ser original.

Encima, en un proyecto Expo SDK 58, Expo Router. La ruta es el archivo, los href van tipados, y los deep links salen de ese árbol en vez de un `linking` config que se desfasa del navegador. Para el logout y para un reset duro uso la referencia imperativa. Router no me prohíbe echar a alguien al root.

Deep links. En iOS, Universal Links: el archivo `apple-app-site-association` servido por el dominio, sin redirects que lo rompan, y el entitlement de associated domains. En Android, App Links: `assetlinks.json` y el intent filter con `autoVerify`. El SHA-256 que hay que publicar es el del

certificado con el que Play firma la app que el usuario instala. Con Play App Signing, ese certificado no es el de tu keystore de upload. Si publicas el SHA del upload, la verificación pasa en el build interno y falla en producción. Debug, release local y Play son tres huellas. Las tres van al archivo si quieres que los tres builds abran el link.

El bug de "navegó dos veces" es de senior. En frío, `getInitialURL` trae el link. En caliente, el evento. Es fácil manejar los dos y empujar la misma pantalla dos veces, o empujar encima de un estado a medio hidratar. Una ref de "este URL ya se consumió" y, con Router, dejar que el sistema lo haga una vez. El parámetro del link es input de usuario. Una URL `transfer?amount=5000&to=...` no arma el pago. Abre la pantalla con los datos visibles y exige confirmación.

Estado del navegador. Persistir el stack tiene sentido en un flujo largo e inocuo, un borrador de configuración. No persisto a alguien dentro de la confirmación de un pago después de que el sistema mató el proceso. Al hacer logout, el root cambia. No hago `navigation.navigate('Login')` encima del stack autenticado, porque el botón atrás devuelve al dinero. Son dos árboles: sesión y no sesión. El bootstrap lee el llavero detrás de un splash y monta uno solo. Nada de flash de la home antes de saber si hay token.

`freezeOnBlur`, vía react-freeze y react-native-screens, congela la pantalla que quedó atrás o en otra pestaña. El componente sigue montado, el estado se queda, los efectos no se reinscriben, y React no renderiza esa pantalla cuando un context cambia en la que está delante. Eso, en una pestaña con un feed y otra que actualiza un store, es la diferencia entre scrollear suave y rerenderizar una lista invisible. No es un unmount. Si necesitas soltar memoria, desmontas con la opción de detach. Si necesitas que un efecto deje de correr, congelar no alcanza: el efecto no se limpia al congelar. Hay que saber cuál de las dos cosas estás pidiendo.

Navegadores anidados de más vuelven opaco el back y el deep link. Un stack por flujo, tabs en la raíz autenticada, modales para lo que de verdad es modal. Y en 0.88, si la app nativa anfitriona es iOS nuevo, el ciclo de vida que hay que conocer es el de `SceneDelegate`. El `AppDelegate` solo ya no le basta a un binario contra el SDK de iOS reciente.

SI INSISTEN

Back en Android

El back lo maneja el stack nativo. Un `BackHandler` propio encima es el bug de "tuve que apretar atrás cuatro veces" o el de "el atrás se comió el del sistema". Lo uso para un caso: cerrar un sheet que no es una pantalla. Y lo limpio en el unmount del efecto.

Tipos

Con las rutas tipadas, un `href` a una pantalla que no existe falla al compilar. Los params son uniones discriminadas, no un `object` genérico. En 0.87 el API de TypeScript estricto es el default: los deep imports de `react-native/Libraries` son error de tipos, y los refs usan tipos de instancia. Lo digo porque un `navigate` con un param mal tipado es justo el bug que el compilador puede cazar si dejamos de huir con `as never`.

FOLLOW-UP

¿Cómo pruebas un Universal Link sin adivinar?

En iOS, el dispositivo real, no el simulador como única prueba, con el dominio de staging también en el entitlement y su propio AASA. Miro la validación de links asociados en el sistema. En Android, `adb` para ver la verificación del intent filter, y compruebo el `assetlinks.json` con el SHA del build que estoy instalando, no con el de otro flavor. Un link que abre el browser es casi siempre archivo mal servido, content-type, redirect, o la huella del certificado equivocada.

FOLLOW-UP

freezeOnBlur y un video o una suscripción en la pantalla de atrás.

Congelar no pausa el video ni cierra el socket. El efecto sigue como lo dejaste porque no hubo cleanup. Si la pantalla de atrás no debe seguir emitiendo, lo pauso con el foco: `useFocusEffect` limpia al perder el foco. Freeze es para no renderizar. El foco es para soltar recursos. Los uso juntos, no como sinónimos.

FRASES QUE TE BAJAN DE SENIOR

- Stack de JS por defecto 'porque se customiza más'.
- Navegar a Login sin cambiar el root.
- Publicar solo el SHA del keystore de upload.
- Creer que freezeOnBlur desmonta la pantalla y limpia efectos.

Cierre. Stack nativo, una sola vez el deep link, dos raíces para la sesión, y congelar no es desmontar.

BLOQUE 04 · 12 MIN

Capa nativa

La spec de un TurboModule, cuándo Expo Modules o Nitro, y cómo se monta RN dentro de una app nativa.

PREGUNTA 09 · MINUTO 34 A 39

La spec de un TurboModule

5 min · Si el código compila mentalmente: Promise frente a sync, listeners, y el conjunto cerrado de tipos.

LO QUE TE PREGUNTAN

Escríbeme la spec TypeScript de un módulo nativo que cifra y que emite un evento. Explícame el camino hasta Kotlin o Swift y qué no puede cruzar la frontera.

VERSIÓN DE TREINTA SEGUNDOS

Una interfaz que extiende TurboModule, registrada con TurboModuleRegistry.getEnforcing. Codegen genera C++, JNI y el protocolo de ObjC. Promise es async. Un boolean de retorno bloquea el hilo de JS. Los eventos piden addListener y removeListeners. No cruzan funciones sueltas, ni Map, ni ciclos. Un secreto no vuelve en el mensaje de error.

RESPUESTA PARA DECIR EN VOZ ALTA

La spec es el contrato, no un comentario. Se parece a esto.

```
import type { TurboModule } from "react-native";
import { TurboModuleRegistry } from "react-native";

export interface Spec extends TurboModule {
  getConstants(): { algorithm: string };
  encrypt(plaintext: string, aad: string): Promise<string>;
  isHardwareBacked(): boolean;
  addListener(eventName: string): void;
  removeListeners(count: number): void;
}

export default TurboModuleRegistry.getEnforcing<Spec>("NativeVault");
```

`encrypt` devuelve Promise, así que el trabajo de Keystore no se sienta en el hilo de JS. `isHardwareBacked` es síncrono a propósito: es un boolean que ya está resuelto, y aun así lo miro dos veces, porque un sync que un día empieza a tocar disco se vuelve un hitch invisible. `getConstants` no es un objeto mágico colgado del módulo. Es un método que Codegen llama para las constantes. `addListener` y `removeListeners` tienen que estar si vas a emitir. El nativo usa ese conteo para saber si queda alguien escuchando. Sin ellos, el evento se ve en JS y se fuga en nativo.

`getEnforcing` tira si el módulo no está linkeado. Eso es lo que quiero en la app. En un test de Jest uso `get` o, mejor, un mock del módulo, porque el test no tiene JNI. El deep import de `react-native/Libraries/...` en 0.87 ya es un error de tipos. Se importa desde la entrada pública.

El camino de build. La spec vive en un archivo que Codegen reconoce. En una librería, `codegenConfig` del package.json dice el nombre, el tipo `modules`, el directorio JS, el package de Android. `pod install` y Gradle generan el C++ abstracto, el JNI y el protocolo de Objective-C o el puente de Swift. Del lado Kotlin implementas la spec generada. Del lado Swift, el protocolo. Autolinking registra el módulo. Si falta

el registro, `getEnforcing` falla al primer acceso, no al arranque: es perezoso. Ese detalle confunde en el bug "la app abre y crashea al tocar cifrar".

Lo que no cruza. El conjunto es cerrado: string, number, boolean, objetos planos descritos como interfaces, arrays, void y Promise. Están los alias de `CodegenTypes` para enteros y para objetos inseguros cuando de verdad hace falta. No pasa una función arbitraria como callback de estilo web. No pasa un `Map`, un `Set`, una clase, un `Date`: el instante viaja como número. No pasa un grafo con ciclos, porque hay que recorrerlo. `undefined` y `null` no son intercambiables en la spec. Para un binario grande no uso base64 por el puente como primera opción. Escribo un archivo y paso el path, o diseño un HostObject si el volumen justifica salir de Codegen y mantener C++.

Los errores. Una excepción nativa rechaza la Promise. El mensaje tiene que ser un código de dominio, no el material de la llave, no el plaintext, no el stack con el alias del keystore. En JS, el `catch` traduce ese código a UI.

Y el evento. En JS, `NativeEventEmitter` sobre el módulo, suscripción en un efecto, cleanup que desuscribe. En nativo, emitir solo si el conteo de listeners es positivo. El nombre del evento es un union en TS, no un string libre repartido por la app.

SI INSISTEN

Sync, la regla

Un método sync está justificado si es barato y si llamarlo desde el render o desde un gesto no puede crecer. `isHardwareBacked` puede serlo. `encrypt` no. `readFile` no. Si dudas, es Promise. Convertirlo después es un cambio de spec y un breaking change para quien ya lo llama.

Vista nativa, que no es lo mismo

Un TurboModule no pinta. Si el nativo es una vista, es un Fabric component: otra spec, `codegenNativeComponent`, un shadow node si el layout depende de algo que Yoga no sabe, y comandos para hablar con la instancia. Mezclar "el módulo del mapa" como TurboModule que de alguna forma devuelve una vista es la respuesta que delata que no has publicado uno.

Expo Modules al lado

Si el módulo es de la app y no una librería para npm, muchas veces ni escribo esta spec a mano. La API de Expo Modules expresa lo mismo en Swift y Kotlin con menos ceremonia y ya cae en New Architecture. La spec de Codegen la enseño porque es el primitivo, y porque una librería que quiere vivir fuera de Expo la tiene que hablar.

FOLLOW-UP

¿Cómo testes este módulo sin dispositivo?

La lógica de cifrado pura, si está en un tipo que puedo correr en JVM o en un test de Swift, se testea allá, con vectores. En JS no testeo el Keystore. Mockeo el módulo en el borde e testeo que la pantalla manda el plaintext correcto y que traduce el código de error. Un test de Jest que importa `NativeVault` de verdad y espera una excepción de JNI no es un test, es un crash. El de dispositivo, uno, en el pipeline de release, cifra y descifra un fixture.

FOLLOW-UP

¿Puedo pasar un ArrayBuffer?

Solo si la versión de Codegen que estoy usando lo tiene como tipo de primera clase y lo he probado en las dos plataformas. Si no, no lo prometo en la entrevista como si fuera un `Uint8Array` de web. Para un payload grande, archivo temporal con path, y borro el archivo en un `finally`. Base64 es el último recurso: infla un tercio y lo pagas en los dos heaps.

FRASES QUE TE BAJAN DE SENIOR

- NativeModules.Vault sin tipos, como el diseño nuevo.
- Un método sync que lee el Keystore.
- Rechazar la Promise con el plaintext o la llave.
- Olvidar removeListeners y perder el conteo nativo.

Cierre. La spec es el producto. Codegen es el build. Sync es la excepción. El error que cruza no lleva material secreto.

PREGUNTA 10 · MINUTO 39 A 43

Expo Modules, Nitro, o bare

4 min · Si dejó atrás la palabra eject, y si no mete Nitro para envolver un solo método async.

LO QUE TE PREGUNTAN

¿Cuándo escribes un Expo Module, cuándo un TurboModule, cuándo miras Nitro, y cuándo de verdad necesitas commitear android e ios?

VERSIÓN DE TREINTA SEGUNDOS

Expo Modules es el default del producto: Swift y Kotlin de alto nivel, New Architecture incluida, y config plugins en prebuild. TurboModules cuando publico una librería fuera de Expo o necesito un Fabric component con shadow node. Nitro cuando el borde se cruza miles de veces por segundo. Commitear las carpetas nativas es la excepción, no el rito de salida. Y el código nativo propio no corre en Expo Go: corre en un dev build.

RESPUESTA PARA DECIR EN VOZ ALTA

La palabra eject ya no describe el sistema. Lo que hay es prebuild y Continuous Native Generation. `npx expo prebuild` genera `android` e `ios`. Puedes no commitearlos y dejar que EAS los genere, que es el modo que prefiero, o commitearlos si hay una razón. Un config plugin modifica manifiesto, Info.plist, Gradle o el Podfile en esa generación. Permisos, un `intent-filter`, un meta-data de un SDK: eso es un plugin, no un proyecto bare.

Para el módulo en sí, la API de Expo Modules es el default. Escribes Swift y Kotlin con un DSL, el tipo cruza, autolinking lo registra, y un módulo escrito así ya funciona en New Architecture. Vive en `modules/` dentro de la app. No hace falta publicar un paquete. Esto cubre el caso normal: leer un SDK de terceros, cifrar, hablar con un hardware, emitir eventos.

Bajo a la spec de TurboModule y Codegen en tres casos. Publico una librería que tiene que consumirse en un proyecto sin Expo. Necesito un método cuyo tipo el DSL todavía no expresa y no quiero

envolverlo en un hack. O la pieza no es un módulo sino una vista con un shadow node de C++ que participa del layout. Ahí el primitivo de RN es el camino corto, aunque el boilerplate sea peor.

Nitro Modules, el sistema de Margelo, entra en la conversación cuando el costo del borde es el producto. Genera bindings JSI con menos ceremonia que Codegen a pelo y está pensado para cruces muy frecuentes. MMKV en sus versiones nuevas va por ahí. No lo agrego para un `encrypt` que se llama cuando el usuario aprieta un botón. Agregar Nitrogen, otra codegen y otra convención al repo tiene un costo de equipo. Lo pago si el profiler dice que el borde duele, no porque la librería esté de moda.

Bare de verdad, carpetas nativas commiteadas y mantenidas a mano, cuando el cambio no cabe en un plugin: un `CMake` que no es el del módulo, una app anfitriona brownfield, o un SDK cuyo instalador edita el proyecto de una forma que un plugin no puede repetir. Incluso ahí, el primer intento es el plugin. Commitear `android` e `ios` significa que cada upgrade de SDK es un merge de proyectos nativos. Eso es un equipo que sabe hacer ese merge, no una medalla.

Y el flujo de desarrollo, que es donde se caen los candidatos. Expo Go del SDK 58 corre el SDK 58 y no carga tu módulo nativo. En cuanto hay código nativo propio, el día a día es un development build, `expo-dev-client`, instalado en el dispositivo. Metro recarga el JS. El nativo nuevo exige recompilar ese dev build. Decir "lo pruebo en Expo Go" con un módulo propio es decir que no lo has corrido.

El orden que uso para decidir, en voz alta: ¿cabe en JS? Entonces no hay módulo. ¿Cabe en un Expo Module? Entonces eso. ¿El borde es el cuello de botella medido? Entonces miro Nitro o un HostObject. ¿El proyecto nativo necesita una edición que el plugin no repite? Entonces bare, y con los ojos abiertos.

SI INSISTEN

Lo que un config plugin no debe hacer

Ser un script imperativo que alguien corre una vez y olvida. Tiene que ser idempotente: correrlo dos veces deja el mismo proyecto. Se testea generando el nativo en CI. Si el plugin depende de un estado que ya estaba editado a mano, no es un plugin, es un recuerdo.

Versiones

El módulo nativo se compila contra el RN del binario. Un OTA no puede añadir el módulo después. Si la pantalla JS llama a `getEnforcing` y el binario viejo no tiene el módulo, esa pantalla tiene que saber decir "actualiza desde la tienda", no reventar. `runtimeVersion` o el fingerprint es lo que evita servirle ese JS al binario viejo. Las dos defensas van juntas: el canal no le manda el update, y el código aguanta el módulo ausente por si un binario raro se cuela.

FOLLOW-UP

¿Monorepo con una app Expo y un paquete nativo interno?

El paquete interno es un Expo Module con su propio `expo-module.config.json`, y la app lo referencia por el workspace. Autolinking lo encuentra si el paquete está en las dependencias, no si solo está en el disco. El dev build se compila desde la app. No intento que Expo Go lo cargue. Y el versionado del módulo sigue al binario: cambiar el nativo es un build de EAS, no un publish de JS.

FOLLOW-UP

Un SDK cierra el paso pidiendo que edites MainApplication a mano.

Eso cabe en un config plugin: hay mods de Expo para el manifiesto, para Gradle y para los archivos de la app. Escribo el mod, lo corro en prebuild limpio y miro el diff. Si el SDK inyecta un archivo binario o un paso de CMake que el mod no puede expresar, ahí sí genero las carpetas y las commiteo, y dejo escrito en el README del módulo por qué ese SDK nos sacó de CNG. La razón queda al lado del código, no en un hilo de Slack.

FRASES QUE TE BAJAN DE SENIOR

- Decir eject como si siguiera existiendo el comando.
- Probar un módulo nativo propio dentro de Expo Go.
- Meter Nitro para un método que se llama una vez por pantalla.
- Commitear android e ios el primer día, por costumbre.

Cierre. JS si alcanza. Expo Module si hace falta nativo. Codegen si es una librería o una vista. Nitro si el profiler lo pide. Bare si el plugin no puede repetir el cambio.

PREGUNTA 11 · MINUTO 43 A 46

React Native dentro de una app nativa

3 min · Si habla de un solo ReactHost, de precargar el runtime, y del contrato de back entre los dos mundos.

LO QUE TE PREGUNTAN

La app ya existe en Swift y Kotlin. Van a meter una superficie de React Native. ¿Cómo la montas y qué costos pones sobre la mesa?

VERSIÓN DE TREINTA SEGUNDOS

Un solo ReactHost para todas las superficies, precargado durante el splash del feature, no en el tap. El nativo es el dueño del back stack y RN es un stack interno con un contrato escrito. Se hablan por un TurboModule del host, no por un global. El OTA solo mueve JS compatible con el RN que el binario anfitrión ya trae. Dos runtimes duplican memoria.

RESPUESTA PARA DECIR EN VOZ ALTA

Brownfield significa que el proceso, la navegación de fondo y la cuenta de release son de la app nativa. RN es una superficie, no el dueño del `Application`.

En iOS, el punto de entrada actual es la factory de React, no el `RCTRootView` que la gente recuerda de 2019. En 0.88 además tienes que hablar el ciclo de `SceneDelegate`, porque un binario contra el SDK de iOS reciente no arranca solo con `AppDelegate`. Cargas una URL de bundle —embebida en release, Metro en dev— y un nombre de módulo. En Android, un `ReactHost` dentro de la Activity o un Fragment. La API de host de la New Architecture es la que se integra. No hay bridge que configurar.

Quiero un solo host para todas las pantallas de RN. Dos instancias son dos Hermes de aplicación, dos registros de módulos y dos árboles. La memoria se nota en la gama media, que es donde el equipo nativo va a juzgar el experimento. El runtime de worklets no cuenta como segundo host.

El costo de arranque se paga una vez y se esconde. Inicializo el host cuando el usuario entra en la zona de la app donde RN es probable, durante un splash que la app nativa ya tenía, no en el frame del tap. El bundle va en bytecode dentro del binario. Mido el TTI de la superficie aparte del TTI de la app. Si los mezclas, nadie sabe quién regresó.

La comunicación tiene spec. El host implementa un módulo, `HostAuth.getSession()`, `HostNavigation.close()`, y RN lo llama como cualquier TurboModule. Los props iniciales sirven para la primera pantalla. Un evento avisa de un cambio de sesión. No cuelgo funciones en `global`. No mando el usuario completo en cada navegación. Los objetos que cruzan son chicos y tipados, por las mismas razones que en cualquier borde JSI.

El back se acuerda por escrito. El botón físico de Android lo ve primero el contenedor nativo. Si RN tiene un stack interno, el host le pregunta "¿consumiste el back?" antes de sacar la Activity. Dos `BackHandler` que no se conocen es el bug de la primera semana. En iOS, el gesto de volver del `UINavigationController` y el gesto interno de RN no se habilitan a la vez.

Y el tren de release deja de ser solo de RN. La versión de React Native es dependencia del binario anfitrión. Subirla es un release de tiendas, coordinado con el equipo nativo. El OTA cubre el JS de esa superficie y nada más, con el `runtimeVersion` atado a ese binario. El equipo anfitrión tiene que saber que un `npm update react-native` no es invisible para ellos.

En CI hay un host de juguete, una app nativa mínima que solo monta la superficie. Los tests de la superficie corren ahí. Si la única forma de probar es compilar el banco entero, nadie prueba.

SI INSISTEN

Tamaño

El costo no es solo Hermes. Son los SDKs que el equipo de RN arrastra: mapas, video, un cliente que duplica el que la app nativa ya tenía. Antes de meter la superficie, miro el tamaño del binario con y sin ella. Un experimento que mete 25 MB para una pantalla de ajustes no sobrevive al comité nativo, y no debería.

Equipo

El contrato del módulo es un archivo de la spec, revisado por los dos lados. El sample host es del mismo PR que cambia la spec. Sin eso, brownfield se convierte en "el equipo de RN rompió el build de iOS" cada semana.

FOLLOW-UP

¿La sesión se loguea en nativo y RN solo la lee?

Sí, si el login ya existe y es nativo. RN no guarda otro token. Pide `HostAuth.getSession()` al host, y el host lee el llavero que ya tenía. El refresh lo hace el host, que es el dueño. Si cada lado refresca, uno invalida al otro. El evento de logout lo emite el host y RN destruye su caché. Query, store y memoria. No queda un saldo en MMKV de una cuenta que ya cerró.

FRASES QUE TE BAJAN DE SENIOR

- Un RCTRootView de tutorial viejo como único plan.
- Dos ReactHost 'para aislar features'.
- global.nativeCallback = ...
- Cargar el runtime en el onPress y llamar a eso instantáneo.

Cierre. Un host, precargado, un contrato tipado, un solo dueño del token y un back que está escrito.

BLOQUE 05 · 8 MIN

Gestos, teclado e insets

Worklets en el hilo de UI y el teclado en Android edge-to-edge, que es donde se caen las listas.

PREGUNTA 12 · MINUTO 46 A 51

Worklets y el snap de un sheet

5 min · Si sabe que Reanimated 4 solo vive en New Architecture, que el plugin de worklets va al final, y que no vuelve a JS en cada frame.

LO QUE TE PREGUNTAN

Explícame un worklet, un shared value y scheduleOnRN. ¿Por qué un PanResponder con el Animated clásico no aguanta un bottom sheet a 120 Hz? Cuéntame el snap.

VERSIÓN DE TREINTA SEGUNDOS

Un worklet corre en el runtime de UI, no en el de React. El shared value se escribe ahí y el estilo animado lo lee sin render. scheduleOnRN, antes runOnJS, vuelve a RN una sola vez, con los argumentos en la misma llamada. PanResponder entrega el dedo a JS: a 8 ms no llega. El snap proyecta la posición con la velocidad y cae al punto más cercano con withSpring, en el worklet.

RESPUESTA PARA DECIR EN VOZ ALTA

Reanimated 4 solo existe sobre New Architecture. La 3 no recibe este runtime. Los worklets se mudaron al paquete `react-native-worklets`. El plugin de Babel es `react-native-worklets/plugin` y va el último. Si también usas React Compiler, el compiler va primero y los worklets al final. `useAnimatedGestureHandler` se eliminó. El gesto es el `Gesture` de Gesture Handler 2.

Un worklet es una función que el plugin convierte en algo serializable y que puede correr en el otro runtime, el que está montado en el UI thread. `useSharedValue` guarda el dato en un sitio que los dos runtimes leen. Escribir `translateY.value` en el hilo de UI no provoca un render de React. `useAnimatedStyle` devuelve un estilo que se actualiza desde ese valor, en el mismo hilo, sin reconciliar.

Los callbacks de `Gesture.Pan()` ya corren como worklets en el UI thread. Si los fuerzas con `runOnJS(true)`, bajan a JS y dejas de poder tratarlos como worklets. Para volver a React uso `scheduleOnRN`, importado de `react-native-worklets`. El API cambió: los argumentos van en la misma llamada, `scheduleOnRN(fn, arg)`. El `runOnJS(fn)(arg)` es el deprecado. Es asíncrono. Lo que cruza tiene que ser shareable: números, strings, objetos planos. No un `setState` cerrado sobre medio árbol, y no cuarenta veces por segundo.

Por eso PanResponder más `Animated` con `useNativeDriver: false` no es un sheet. PanResponder entrega el touch al hilo de JS. Desde ahí mueves un valor y cruzas para pintar. `useNativeDriver: true` solo anima una lista blanca, transform y opacidad, y no puede decidir en cada frame un clamp o un snap con lógica tuya. A 120 Hz tienes unos 8,3 ms. El primer render que se te cruce —una lista, un teclado— separa el dedo de la vista.

El snap, dicho para poder teclearlo después. Tres puntos, por ejemplo 80, 320 y 560. En `onStart` guardo `startY`. En `onUpdate` sumo `translationY`. Si me paso del primero o del último, aplico resistencia, un factor de 0,3, para que el dedo siga y la vista se quede. En `onEnd` proyecto: posición más velocidad por un coeficiente chico. Elijo el punto con menor distancia a esa proyección.

`translateY.value = withSpring(destino, { velocity })`. Si el destino es el cerrado, `scheduleOnRN(close)` una vez. Cerrar desmonta en React. Mover no.

`withSpring` en la versión 4 cambió el criterio de reposo a `energyThreshold`, y `duration` es perceptual: el tiempo real es alrededor de 1,5 veces ese número. Si porto un spring de la 3 que "se sentía bien", no asumo los defaults. Están exportados como `Reanimated3DefaultSpringConfig` para el que los necesite.

Dos trampas de setup. `GestureHandlerRootView` envuelve la app una vez. Y el import de `react-native-gesture-handler` entra en el entry antes que el resto, por el side effect que registra los gestos. Un sheet que funciona en un ejemplo y no en la app casi siempre es una de esas dos, o un padre que se quedó con el gesto.

Lo que no hago en el worklet: llamar a la red, leer el store de React, loguear en cada frame, ni capturar un objeto que muta en JS. Si una función de ayuda vive en otro archivo y la llamo desde el worklet, esa función también es un worklet. Si se me olvida, el error es oscuro y el frame cae a JS.

SI INSISTEN

Qué animo

`transform` y `opacity`. Si el sheet empuja contenido, el contenido de abajo no debería relayoutear en cada frame. Le dejo el hueco con un spacer de altura fija o animo su transform. Animar `height` del sheet es la forma lenta, y se siente "nativa" solo en el simulador.

Teclado contra el sheet

No envuelvo el sheet en `KeyboardAvoidingView`. La altura del teclado entra como otro shared value, desde `react-native-keyboard-controller`, y el snap la resta. Si el teclado abre y el snap se recalcula en JS con `setState`, volviste al problema del principio.

Flags

Si después de subir a New Architecture el scroll con cabeceras animadas parpadea, no culpo al gesto. Reanimated documenta regresiones de commit en la New Architecture y flags de la serie 4 para mitigarlas. Los leo en la guía de la versión instalada. Reanimated 4.2 y posteriores movieron varias de esas optimizaciones. Citar un flag de un issue de 0.81 sin mirar el que hay en el lockfile es adivinar.

FOLLOW-UP

¿El shared value sobrevive a un re-render de React?

Sí. Esa es la gracia. El componente puede pintar por otra razón y `translateY` sigue donde el dedo lo dejó, porque no vive en `useState`. Lo que no sobrevive mal hecho es recrear el gesto de forma que `onStart` vuelva a leer un valor viejo capturado en un closure de JS. Por eso el punto de partida se copia a otro shared value en `onStart`, dentro del worklet, no a una variable `let` del render.

FOLLOW-UP

¿CSS animations de Reanimated reemplazan esto?

Reemplazan la animación de montaje: entrar, salir, un fade. No reemplazan un gesto. El dedo es un shared value actualizado por el pan. Puedo mezclar las dos en la misma pantalla. No puedo hacer snap con una transition de CSS.

FRASES QUE TE BAJAN DE SENIOR

- useAnimatedGestureHandler como el API actual.
- runOnJS(fn)(arg) como la forma vigente.
- setState dentro de onUpdate.
- El plugin de worklets en cualquier posición de Babel.

Cierre. El dedo no sale del UI thread. React se entera una vez, cuando el gesto ya decidió.

PREGUNTA 13 · MINUTO 51 A 54

Teclado, edge-to-edge e insets

3 min · Si sabe que adjustResize ya no salva una app edge-to-edge, y que el inset y el teclado no se suman.

LO QUE TE PREGUNTAN

El input queda detrás del teclado en Android 15 y el botón de abajo flota mal con el gesto de navegación. ¿Qué está mal del enfoque clásico?

VERSIÓN DE TREINTA SEGUNDOS

Android edge-to-edge dibuja detrás de las barras del sistema. adjustResize ya no encoge la ventana como antes: el teclado se superpone y hay que consumir el inset del IME. KeyboardAvoidingView alrededor de una lista virtualizada la rompe. Uso react-native-keyboard-controller, que sigue al teclado en el hilo de UI. El safe area sale de insets, no de un 44 fijo, y no se lo sumo al teclado.

RESPUESTA PARA DECIR EN VOZ ALTA

El enfoque clásico era `KeyboardAvoidingView` con `behavior="padding"` en iOS y `undefined` en Android, confiando en `windowSoftInputMode=adjustResize`. Eso asumía que el sistema encogía la ventana y tu layout se enteraba. Con edge-to-edge, que en Android 15 es el modo en el que la app dibuja detrás de la status bar y de la barra de navegación, la ventana ya no se encoge así. El teclado se dibuja encima. Si nadie consume el inset del IME, el input queda debajo. `adjustPan` es peor en una lista: mueve la ventana entera y descoordina la virtualización.

La herramienta que uso es `react-native-keyboard-controller`. `KeyboardProvider` arriba, y la altura del teclado como un valor que puede manejar Reanimated, en el hilo de UI, así el sheet y el composer acompañan al teclado en el mismo frame. Envolver una `FlashList` en `KeyboardAvoidingView` es el bug: la lista recalcula la ventana, salta, y a veces dispara `onEndReached`. Al composer le pongo un spacer inferior del alto del teclado, o uso la integración de la lista si la versión que tengo la trae bien. El footer de la lista no es un `padding` mágico en el padre.

Insets. `react-native-safe-area-context`, `useSafeAreaInsets()`, y el provider tiene que envolver también el árbol de un modal nativo si ese modal tiene su propia jerarquía. No hardcodeo 44 ni 24. En Android el status bar, la barra de navegación y el cutout son insets distintos. En horizontal, el notch se vuelve inset izquierdo o derecho. El botón de "pagar" que está fijo abajo lleva el inset inferior cuando el teclado está cerrado. Cuando el teclado está abierto, ese inset inferior ya no aplica: lo reemplaza la altura del teclado. Sumar los dos es el hueco enorme que todo el mundo ha visto y que se diagnostica en diez segundos si sabes que existe.

En iOS, `keyboardDismissMode="interactive"` en la lista, y el composer que sigue el frame del teclado, no un `keyboardWillShow` que hace `setState` con la altura y pierde el frame del gesto. `InputAccessoryView` cuando el accesorio es de verdad del sistema.

Y las props que ya no recomiendo. En el salto de SDK 58 se fueron, entre otras, `backgroundColor`, `translucent` y `networkActivityIndicatorVisible` de `StatusBar`. El color de las barras, en edge-to-edge, es del sistema de insets y de `expo-status-bar` o de la librería de edge-to-edge, no de pintar una view detrás y esperar. `ImageBackground` también quedó deprecado a favor de una `View` con una `Image` absoluta. Son detalles chicos que delatan la fecha del código que escribes.

`SafeAreaView` del core de React Native no es el que uso. El de la comunidad, el context, sí, porque entiende el provider y los modales.

SI INSISTEN

Modal

Un modal a pantalla completa presentado por `react-native-screens` puede tener un safe area distinto al de la pantalla de abajo. Si el provider no está, los insets llegan en cero y el contenido se mete en el notch solo en esa pantalla. Es un bug que en el simulador con notch chico casi no se ve. Lo pruebo en un dispositivo con home indicator y en un Android con barra de tres botones y en uno con barra de gestos. Son dos insets inferiores distintos.

Texto que crece

Un composer de chat cuyo alto crece con el texto no puede empujar la lista con layout en cada línea si además hay teclado animándose. El alto del composer es estado de React, sí, porque es texto. El desplazamiento del teclado es un shared value. No los meto en el mismo `setState`.

FOLLOW-UP

¿adjustResize entonces no se usa nunca?

No como estrategia de producto en una app edge-to-edge. Puede quedar en un proyecto viejo que no dibuja detrás de las barras, y ahí el comportamiento clásico todavía describe la ventana. En cuanto activas edge-to-edge, asumo que el IME es un inset y lo trato como tal. Si un fabricante raro no reporta el inset, lo veo en el dispositivo, no lo tapo con un padding de 300.

FRASES QUE TE BAJAN DE SENIOR

- Un paddingBottom de 44 para todos los teléfonos.
- KeyboardAvoidingView rodeando FlashList.
- Sumar safe area inferior y altura de teclado.
- StatusBar con translucent como la receta de edge-to-edge.

Cierre. El teclado es un inset que se anima en el hilo de UI. No se suma al home indicator. No envuelve a la lista.

BLOQUE 06 · 6 MIN

Tests, release y seguridad

Qué testear, qué puede viajar por OTA y qué secreto no puede vivir en el bundle.

PREGUNTA 14 · MINUTO 54 A 57

Qué testea un senior

3 min · Si testea comportamiento, si deja la red en el borde, y si no promete 400 tests de Detox.

LO QUE TE PREGUNTAN

¿Cómo partes los tests de una app de React Native? ¿Qué no mockeas? ¿Maestro o Detox?

VERSIÓN DE TREINTA SEGUNDOS

La lógica pura, paginación y dinero, en Jest sin render. La pantalla con React Native Testing Library, por rol y por texto, con los providers de verdad y la red stubbeada. No mockeo el hook y el componente a la vez. Maestro cubre los cinco flujos que cobran, en el release. Detox solo si necesito sincronización que la caja negra no tiene. InteractionManager no se testea porque ya no existe.

RESPUESTA PARA DECIR EN VOZ ALTA

La pirámide que defiendo es aburrida, y eso es lo que quiero.

Abajo, Jest puro, sin renderizar React. El reducer de la paginación, el cálculo del snap, el formato de dinero, la decisión de si un pago entra en la bandeja. Son milisegundos y fallan por la razón correcta. Si el snap está mal, no quiero un test de UI para descubrirlo.

En el medio, `@testing-library/react-native`. Renderizo la pantalla, busco por rol o por texto, y disparo con `userEvent`. El testID es el último recurso, cuando no hay nombre accesible, y si lo uso me pregunto por qué esa vista no es accesible. Un `renderWithApp` monta los providers: QueryClient con `retry: false`, safe area con insets fijos, navegación. La red no se mockea módulo por módulo. Se stubbea en el fetcher o con MSW, y la pantalla pasa por su estado de carga, su vacío, su error y su dato. Esos cuatro los quiero ver en el test de la pantalla que cobra.

Lo que no mockeo es la unidad bajo prueba. Si mockeo el hook, mockeo el componente hijo y después afirmo que el mock fue llamado, no probé nada. Tampoco mockeo Query para "simular que hay data" si lo que estoy probando es que la pantalla muestra el error: un QueryClient de test y un fetcher que rechaza lo hacen de verdad. Lo que sí mockeo son los módulos nativos que revientan Jest: el mock oficial de Reanimated, el de Gesture Handler, el de screens. Sin eso el preset no arranca. El preset de Jest de las versiones nuevas es `@react-native/jest-preset`.

Snapshots de pantallas enteras, no. Se pudren y nadie lee el diff. Un snapshot chico de un primitivo del design system puede vivir.

Los timers se fingean y se avanzan. Si dejo un interval real, el test depende de la máquina. React 19 y `act` se llevan mal con un `setState` que llega tarde: lo adelanto en el test, no lo ignoro con un comentario.

Arriba, poco. Maestro es caja negra, YAML, corre contra un release y en CI duele menos que Detox. Lo uso para el camino que no podemos romper: entrar, ver el saldo, preparar un pago, confirmar, salir. Como no sabe cuándo Hermes está idle, las esperas son explícitas y cortas, sobre texto que la UI

muestra, no sobre `sleep` de ocho segundos. Detox es caja gris, se sincroniza con el runtime y es más potente, y también más caro de mantener sobre New Architecture. Lo traigo si un flujo no se puede afirmar en negro. No prometo una suite de cientos de tests end to end. Esa suite se salta, se pone en cuarentena y deja de ser una red.

Coverage como objetivo, no. Puedo tener 90 % sin probar el pago. Prefiero la lista de comportamientos. Y un test de Reassure en la lista que nos importa, para que un render de más falle el PR. Eso atrapa la clase de bug del live coding mejor que otro snapshot.

SI INSISTEN

Accesibilidad como efecto del test

Si el test no encuentra el botón por el nombre, el lector de pantalla tampoco. Arreglo el `accessibilityLabel` o el texto, no le pongo testID para salir del paso. En una entrevista de senior esto cuenta doble: estás diciendo cómo pruebas y cómo queda la app para alguien que no la ve.

Datos

El test de UI no pega al backend de staging. El de Maestro, si corre en CI, pega a un entorno controlado o a un build con el fetcher de fixture. Un test e2e que depende del saldo real de una cuenta compartida falla los viernes y enseña al equipo a ignorarlo.

FOLLOW-UP

¿Cómo pruebas el hook de paginación?

Sin render. Le paso un `fetchPage` falso que anota el cursor y que puedo retrasar. El caso que me importa: disparo un refresh mientras el append anterior no resolvió, resuelvo el append tarde y afirmo que sus ítems no pisaron el refresh. Ese es el bug de carrera. El segundo caso: `loadMore` sin cursor no aborta el refresh en vuelo. El tercero: un error deja los ítems que ya estaban. Si esos tres pasan, la pantalla solo tiene que cablear el hook.

FRASES QUE TE BAJAN DE SENIOR

- Cobertura del 100 % como definición de terminado.
- Mockear useQuery en el test de la pantalla que muestra el error.
- Una suite enorme de Detox que nadie corre en el PR.
- Snapshots de pantallas como red principal.

Cierre. Lógica pura abajo, comportamiento de la pantalla en el medio, cinco flujos de dinero arriba. El mock vive en la frontera nativa y en la red, no en el centro.

PREGUNTA 15 · MINUTO 57 A 60

OTA, secretos y crashes

3 min · Si ata el update al runtime, si no esconde secretos en el bundle, y si symbolica las dos clases de crash.

LO QUE TE PREGUNTAN

Cierra con el pipeline de release. ¿Qué puede ir en un OTA? ¿Dónde viven los secretos? ¿Cómo separas un crash de JS de uno nativo?

VERSIÓN DE TREINTA SEGUNDOS

EAS Build firma el binario. EAS Update mueve JS, firmado, y solo hacia un binario cuyo runtimeVersion coincide. No mete módulos nativos ni un bytecode de otro Hermes. El secreto no va en el bundle: el token va al llavero. Pinning solo si el modelo de amenaza paga el costo de rotar pines. Sentry o Crashlytics para el nativo, el SDK de JS para JS, source maps distintos por release y por update.

RESPUESTA PARA DECIR EN VOZ ALTA

Tres perfiles, y no los mezclo. Development es el dev client, con Metro. Preview es un binario interno, el que QA instala. Production es el que firman las tiendas. EAS Build produce esos binarios. EAS Submit los manda a TestFlight y a Play. EAS Update publica JS por encima.

`runtimeVersion` decide quién puede recibir el update. El fingerprint, que cambia cuando cambian los inputs nativos, es la política que prefiero porque no depende de que alguien se acuerde de subir un string. Un string manual también sirve si el equipo es disciplinado y el string cambia en el mismo PR que el nativo. Lo que no sirve es un runtime fijo de por vida: tarde o temprano un update con un módulo nuevo le llega a un binario que no lo tiene, y `getEnforcing` tira en la mano del usuario.

El update va firmado. La app lleva la clave pública, `expo-updates` rechaza lo que no firma la privada. Un OTA es ejecución remota de código por diseño. Sin firma, quien pueda escribir en el canal puede ejecutar JS en tus usuarios. Eso, en una fintech, no es un endurecimiento opcional.

Qué no entra en el OTA. Un módulo nativo, un permiso de Android, un entitlement, un cambio de Hermes, un bytecode compilado con otro `hermesc`. El icono y los assets que viven en el catálogo nativo tampoco. Si el permiso es nuevo, es release de tienda. El JS que llama a ese módulo tiene que aguantar el binario viejo o no ser servido, las dos cosas.

Secretos. El bundle se descarga en cada teléfono y el bytecode se descompila. Una API key privada dentro del repo es una API key filtrada, aunque esté en `.hbc`. Al cliente le tocan identificadores públicos. El token sale del servidor de auth y se guarda en SecureStore. El refresh rota. En Android, las pantallas de dinero llevan `FLAG_SECURE` para que no salgan en recientes ni en un screenshot casual. iOS no tiene ese flag global: se pone un blur en el switcher cuando la app va a background. No prometo que eso detiene a un dispositivo comprometido. Digo qué cubre.

Pinning de certificado. Se puede, con un módulo nativo o con el pinner de OkHttp, y se opera mal: el día que rotas el certificado sin un pin de respaldo, apagas la app. Lo hago si el modelo de amenaza es una red hostil y el equipo tiene el runbook de rotación. Si no, TLS más la transparencia de certificados es la base, y lo digo así, sin disfrazar la pereza de arquitectura ni el pinning de gratis.

Crashes. Un `ErrorBoundary` y el handler global de JS no ven el crash nativo, ni un fatal de Hermes que tira el proceso. Hace falta el SDK de JS y el de nativo, Sentry o Crashlytics. Desde SDK 57, el observability de Expo también registra crashes nativos, además de errores de JS. Los source maps del binario y los del update son dos artefactos. Si subes solo los del release de tienda, el OTA de la semana siguiente symbolica mal y persigues una línea que no existe. Lo pruebo: fuerzo un throw en un update y miro que el stack tenga nombre de archivo.

Play App Signing, otra vez, porque pega con los App Links: la huella pública es la del certificado de la app, no la del upload. R8 en Android puede borrar un TurboModule si las reglas de keep no cubren el JNI. Lo veo en un release minificado, no solo en debug. Y bitcode ya no existe: no lo menciono como un paso del release, porque quien lo menciona no ha subido una app en años.

El cierre que me gusta dejar en la mesa: el binario es el contrato de lo nativo, el update es JS firmado y compatible, el secreto no viaja en ninguno de los dos artefactos, y un crash sin símbolo no es un crash, es un rumor.

SI INSISTEN

Rollback

`expo-updates` puede volver atrás si el JS nuevo crashea al arrancar. Eso cubre el bytecode incompatible y el error de import en el root. No cubre un bug de lógica que corrompe datos un minuto después. Para eso, el update se publica en un porcentaje, se mira el crash-free, y se detiene. Publicar al 100 % el viernes a las seis es la decisión que el pipeline debería hacer incómoda.

Lo que le pregunto yo al entrevistador, si queda un minuto

En qué versión están de verdad, si el OTA está firmado, cómo miden el frame de la pantalla que más duele, y si `android` e `ios` están generados o commiteados. Las cuatro respuestas me dicen el trabajo.

FOLLOW-UP

Un update rompió el arranque en el 10 % de los usuarios. ¿Qué haces en la primera hora?

Paro el rollout. Si la recuperación automática ya devolvió a la gente al update anterior, confirmo en el dashboard que el crash de arranque baja. Si no volvió, publico un update correctivo con el mismo runtime, que es JS, no un binario, siempre que el binario pueda ejecutarlo. No meto un release de tienda en esa hora salvo que el bug sea nativo. Después miro por qué el crash de arranque no se probó en el preview. Falta un smoke de Maestro que abre la app con el update aplicado y ve la home.

FOLLOW-UP

¿Dónde pones la clave que firma los updates?

En el proveedor de secretos de CI, no en el repo, no en el bundle, no en una variable con prefijo público que el build de JS incrusta. El binario solo lleva la pública. Rotarla es un release de tienda, porque la pública está dentro. Por eso la rotación de esa clave es un evento, no un `git mv`.

FRASES QUE TE BAJAN DE SENIOR

- Una API key privada en el bundle 'ofuscada'.
- runtimeVersion que nunca cambia.
- OTA sin firma en una app que mueve dinero.
- Un solo source map para el binario y para todos los updates.
- Prometer que el pinning es obligatorio y gratis.

Cierre. Binario para lo nativo, update firmado para JS, llavero para el secreto, y símbolos para las dos clases de crash.

30 MINUTOS

Live coding

Habla el diagnóstico antes de importar la librería. El código inicial es el que te sientan delante. La solución es una forma senior de dejarlo, no la única.

EJERCICIO 01 · 12 MIN

El feed que se repinta entero

ENUNCIADO

Te sientan delante de este `FeedScreen`. Son 5.000 movimientos. Cada vez que el reloj avanza, el JS thread se arrodilla, y al cambiar el filtro las filas muestran el logo y el estado de otra fila.

Tienes unos doce minutos. No te piden un rediseño de la app. Te piden que el filtro siga funcionando, que el reloj siga visible, y que una fila no herede el estado de otra.

Habla mientras escribes. El entrevistador quiere oír el diagnóstico antes que el import de FlashList.

LO QUE EL ENTREVISTADOR MIRA

- Separas el reloj del dueño de la lista antes de hablar de memo.
- El keyExtractor deja de ser el índice, y dices por qué.
- Entiendes que FlashList v2 recicla la instancia: el useState de la fila se resetea con el id.
- No clonas los ítems al filtrar. Conservas la referencia que vino de Query.
- La imagen tiene tamaño y recyclingKey. Mencionas que el compiler no arregla una prop que sí cambia.

CÓDIGO INICIAL

```
import { useEffect, useState } from "react";
import { FlatList, Image, Pressable, Text, View } from "react-native";

type Tx = {
  id: string;
  title: string;
  kind: "card" | "transfer";
  logo: string;
};

export function FeedScreen({ items }: { items: Tx[] }) {
  const [filter, setFilter] = useState<"all" | "card" | "transfer">("all");
  const [now, setNow] = useState(() => Date.now());
  const [openId, setOpenId] = useState<string | null>(null);

  useEffect(() => {
    const id = setInterval(() => setNow(Date.now()), 1000);
    return () => clearInterval(id);
  }, []);

  const filtered = items
    .filter((tx) => filter === "all" || tx.kind === filter)
    .map((tx) => ({ ...tx }));

  return (
    <View style={{ flex: 1 }}>
      <Text>{new Date(now).toLocaleTimeString()}</Text>
      <Pressable onPress={() => setFilter(filter === "card" ? "all" : "card")}>
        <Text>Solo tarjetas</Text>
      </Pressable>
      <FlatList
        data={filtered}
        keyExtractor={(_, index) => String(index)}
        renderItem={({ item }) => (
          <Pressable
            onPress={() => setOpenId(item.id)}
            style={{ padding: 16, backgroundColor: openId === item.id ? "#efe7ff" : "#fff" }}
          >
```

```
            <Image source={{ uri: item.logo }} style={{ width: 36, height: 36 }} />
            <Text>{item.title}</Text>
            <Text>{openId === item.id ? "abierto" : "cerrado"}</Text>
          </Pressable>
        )}
      />
    </View>
  );
}
```

DIAGNÓSTICO, ANTES DE REESCRIBIR

Qué está mal, en el orden en que lo diría

- El intervalo vive en el mismo componente que la lista. Cada segundo se ejecuta `FeedScreen`, se filtra, se clona y FlatList vuelve a pedir celdas.
- El `.map` que copia el ítem rompe la identidad. Aunque la fila estuviera memoizada, todas las props cambiaron.
- `keyExtractor` por índice. Al filtrar, la fila 3 pasa a ser otro movimiento y conserva el estado visual de la anterior. Con reciclaje esto empeora.
- `renderItem` cierra sobre `openId`. Abrir una fila re-renderiza toda la ventana, porque cada celda recibe un boolean nuevo o, peor, el id abierto.
- `Image` de React Native no recicla el bitmap. En una lista larga compite con el main thread.
- El estilo inline es el menor de los problemas. No empieces por ahí.

El estado `openId` en el padre, además, no escala: es un dato por fila guardado fuera de la fila. Con reciclaje hay que decidir quién lo posee.

SOLUCIÓN

```
import { memo, useEffect, useState } from "react";
import { Pressable, Text, View, StyleSheet } from "react-native";
import { FlashList } from "@shopify/flash-list";
import { Image } from "expo-image";

type Tx = {
  id: string;
  title: string;
  kind: "card" | "transfer";
  logo: string;
};

function Clock() {
  const [now, setNow] = useState(() => Date.now());
  useEffect(() => {
    const id = setInterval(() => setNow(Date.now()), 1000);
    return () => clearInterval(id);
  }, []);
  return <Text style={styles.clock}>{new Date(now).toLocaleTimeString()}</Text>;
}

const Row = memo(function Row({ item }: { item: Tx }) {
  const [open, setOpen] = useState(false);
  const [seenId, setSeenId] = useState(item.id);
  if (seenId !== item.id) {
    setSeenId(item.id);
    setOpen(false);
  }

  return (
    <Pressable
      onPress={() => setOpen((value) => !value)}
```

```
      style={[styles.row, open && styles.rowOpen]}
    >
      <Image
        source={{ uri: item.logo }}
        style={styles.logo}
        recyclingKey={item.id}
        cachePolicy="memory-disk"
        contentFit="cover"
      />
      <View>
        <Text>{item.title}</Text>
        <Text>{open ? "abierto" : "cerrado"}</Text>
      </View>
    </Pressable>
  );
});

export function FeedScreen({ items }: { items: Tx[] }) {
  const [filter, setFilter] = useState<"all" | "card" | "transfer">("all");
  const filtered = items.filter((tx) => filter === "all" || tx.kind === filter);

  return (
    <View style={styles.screen}>
      <Clock />
      <Pressable
        onPress={() => setFilter((value) => (value === "card" ? "all" : "card"))}
        style={styles.chip}
      >
        <Text>{filter === "card" ? "Ver todo" : "Solo tarjetas"}</Text>
      </Pressable>
      <FlashList
        data={filtered}
        keyExtractor={(item) => item.id}
        renderItem={({ item }) => <Row item={item} />}
      />
    </View>
  );
}

const styles = StyleSheet.create({
  screen: { flex: 1 },
  clock: { padding: 12 },
  chip: { paddingHorizontal: 12, paddingBottom: 8 },
  row: { flexDirection: "row", gap: 12, padding: 16 },
  rowOpen: { backgroundColor: "#efe7ff" },
  logo: { width: 36, height: 36, borderRadius: 18 },
});
```

QUÉ DECIR MIENTRAS LO ESCRIBES

Lo que voy diciendo mientras lo escribo

Primero saco el reloj. `Clock` es el único que se suscribe al segundo. `FeedScreen` ya no renderiza cada tick. Esto solo, sin cambiar de lista, quita el peor bug. Si el compiler está encendido igual lo quiero así: el compiler no le impide a FlatList ejecutar `renderItem` cuando el padre renderiza, y aquí el padre dejaba de tener motivo.

Después dejo de clonar. `filter` devuelve los mismos objetos que vinieron de la caché. Si Query tiene structural sharing y el refetch no cambió un movimiento, esa fila conserva la referencia.

El abierto o cerrado baja a la fila. Ya no hace falta que todas escuchen `openId`. El patrón `seenId !== item.id` y luego `setState` durante el render es el reset oficial de React cuando la identidad cambia sin unmount. Lo digo en voz alta porque es justo el caso del reciclaje: FlashList v2 reutiliza la instancia. Una `key={item.id}` en `Row` también serviría, y le advierto al entrevistador que una key que cambia anula

parte del reciclaje. El reset durante el render conserva la instancia y tira el estado. Es el compromiso que quiero.

`keyExtractor` pasa a ser `item.id`. Cuento el bug del índice en una frase: al filtrar, la posición 3 es otro movimiento.

La imagen se va a `expo-image` con tamaño fijo y `recyclingKey`. Sin tamaño, el decodificador trabaja de más en el main thread y el jank ni siquiera es de JS. Por eso mirar solo el profiler de Hermes habría mentido.

FlashList v2 no lleva `estimatedItemSize`. Si el entrevistador lo pide, está recordando la v1, y lo digo con cuidado.

No envuelvo el chip en `memo`. No es el costo. Y no subo `windowSize` para tapar el síntoma.

TRAMPAS

- Poner memo en Row y dejar el reloj y el clone en el padre. El memo no tiene nada estable que comparar.
- key={item.id} en la fila sin decir que eso desmonta y pierde el beneficio de reciclar.
- Guardar open en un objeto mutado sobre el ítem. Rompes la caché de Query y el siguiente refetch pisa el flag.
- Seguir usando el índice como key 'porque los ids a veces faltan'. Si faltan, el bug es del API, y se arregla ahí.
- Animated o Reanimated en cada fila para el color de seleccionado. Es un boolean de estilo.

EJERCICIO 02 · 10 MIN

Paginación que no pisa una carrera

ENUNCIADO

Escribe `useCursorPagination`. La pantalla de movimientos pide páginas con cursor. El usuario puede llegar al final mientras jala para refrescar. La respuesta lenta de la página vieja no puede reemplazar el refresh, ni puede vaciar la lista, ni puede duplicar un id.

Diez minutos. Firma, estados y el efecto de la pantalla. Los tests te los cuento al final: no hace falta dejarlos verdes, hace falta decir cuáles son.

LO QUE EL ENTREVISTADOR MIRA

- El early return de loadMore ocurre antes de abortar lo que ya está en vuelo.
- Hay un id de generación además del AbortController, porque no controlas que el fetcher respete la señal.
- Refresh no pone items en vacío.
- Un error conserva lo que ya se había cargado.
- Append deduplica por id.
- fetchPage no está en las dependencias de forma que el hook se reinicie en cada render.

CÓDIGO INICIAL

```
import { useCallback, useState } from "react";

type Page<T> = { items: T[]; nextCursor: string | null };

export function useCursorPagination<T>(
  fetchPage: (cursor: string | null) => Promise<Page<T>>,
) {
  const [items, setItems] = useState<T[]>([]);
  const [cursor, setCursor] = useState<string | null>(null);
  const [loading, setLoading] = useState(false);

  const refresh = useCallback(async () => {
    setLoading(true);
    setItems([]);
    const page = await fetchPage(null);
    setItems(page.items);
    setCursor(page.nextCursor);
    setLoading(false);
  }, [fetchPage]);

  const loadMore = useCallback(async () => {
    if (loading) return;
    setLoading(true);
    const page = await fetchPage(cursor);
    setItems((prev) => [...prev, ...page.items]);
    setCursor(page.nextCursor);
    setLoading(false);
  }, [cursor, fetchPage, loading]);

  return { items, loading, refresh, loadMore };
}
```

DIAGNÓSTICO, ANTES DE REESCRIBIR

Las carreras, dichas antes de tocar el teclado

- `refresh` vacía `items` al empezar. Si falla la red, el usuario se queda sin los movimientos que ya tenía. Y durante el fetch la lista parpadea en vacío.
- `loadMore` y `refresh` comparten un boolean. No impiden que las dos promesas resuelvan. La que responda última gana, aunque sea la página vieja.
- No hay `AbortSignal`. Aunque lo hubiera, un fetcher que lo ignora igual aplicaría el resultado. Hace falta un número de generación.
- `loadMore` usa el cursor del closure. Un refresh a mitad deja ese closure con el cursor anterior, y además este código ni siquiera se niega a pedir cuando el cursor es `null`: pediría la primera página y la concatenaría al final.
- `fetchPage` inline en la pantalla cambia cada render y rearma los callbacks.

El arreglo no es un `loading` más estricto. Es ignorar resultados viejos y no destruir el estado bueno.

SOLUCIÓN

```
import { useCallback, useEffect, useRef, useState } from "react";

export type Page<T> = {
  items: T[];
  nextCursor: string | null;
};

export type PaginationStatus =
  | "idle"
  | "loading"
  | "refreshing"
  | "loadingMore"
  | "error";

type State<T extends { id: string }> = {
  items: T[];
  status: PaginationStatus;
  error: Error | null;
  hasLoaded: boolean;
};

function mergeById<T extends { id: string }>(current: T[], incoming: T[]) {
  const seen = new Set(current.map((item) => item.id));
  const extra = incoming.filter((item) => !seen.has(item.id));
  return extra.length === 0 ? current : [...current, ...extra];
}

export function useCursorPagination<T extends { id: string }>(
  fetchPage: (cursor: string | null, signal: AbortSignal) => Promise<Page<T>>,
) {
  const [state, setState] = useState<State<T>>({
    items: [],
    status: "idle",
    error: null,
    hasLoaded: false,
  });
  const fetchRef = useRef(fetchPage);
  const cursorRef = useRef<string | null>(null);
  const generation = useRef(0);
  const abortRef = useRef<AbortController | null>(null);

  useEffect(() => {
    fetchRef.current = fetchPage;
  }, [fetchPage]);

  const run = useCallback(async (mode: "replace" | "append") => {
    const cursor = mode === "append" ? cursorRef.current : null;
```

```
    if (mode === "append" && cursor === null) return;

    const requestId = ++generation.current;
    abortRef.current?.abort();
    const controller = new AbortController();
    abortRef.current = controller;

    setState((prev) => ({
      ...prev,
      error: null,
      status:
        mode === "append"
          ? "loadingMore"
          : prev.items.length > 0
            ? "refreshing"
            : "loading",
    }));

    try {
      const page = await fetchRef.current(cursor, controller.signal);
      if (requestId !== generation.current) return;
      cursorRef.current = page.nextCursor;
      setState((prev) => ({
        items: mode === "append" ? mergeById(prev.items, page.items) : page.items,
        status: "idle",
        error: null,
        hasLoaded: true,
      }));
    } catch (caught) {
      if (controller.signal.aborted) return;
      if (requestId !== generation.current) return;
      const error = caught instanceof Error ? caught : new Error("Fallo al cargar");
      setState((prev) => ({
        ...prev,
        status: "error",
        error,
      }));
    }
  }, []);

  const refresh = useCallback(() => run("replace"), [run]);
  const loadMore = useCallback(() => run("append"), [run]);

  const abort = useCallback(() => {
    generation.current += 1;
    abortRef.current?.abort();
  }, []);

  return {
    ...state,
    refresh,
    loadMore,
    abort,
    hasMore: cursorRef.current !== null,
  };
}

export function MovementsScreen() {
  const pagination = useCursorPagination(fetchMovements);

  useEffect(() => {
    void pagination.refresh();
    return () => pagination.abort();
  }, [pagination.abort, pagination.refresh]);

  if (!pagination.hasLoaded && pagination.status === "loading") {
    return <LoadingList />;
```

```
  }
  if (!pagination.hasLoaded && pagination.status === "error") {
    return <FullScreenError onRetry={() => void pagination.refresh()} />;
  }
  if (pagination.hasLoaded && pagination.items.length === 0) {
    return <EmptyMovements />;
  }

  return (
    <MovementList
      items={pagination.items}
      refreshing={pagination.status === "refreshing"}
      onRefresh={() => void pagination.refresh()}
      onEndReached={() => void pagination.loadMore()}
      footer={
        pagination.status === "error"
          ? "No se pudo cargar más"
          : pagination.status === "loadingMore"
            ? "Cargando"
            : null
      }
    />
  );
}

declare function fetchMovements(
  cursor: string | null,
  signal: AbortSignal,
): Promise<Page<{ id: string }>>;
declare function LoadingList(): React.JSX.Element;
declare function FullScreenError(props: { onRetry: () => void }): React.JSX.Element;
declare function EmptyMovements(): React.JSX.Element;
declare function MovementList(props: {
  items: { id: string }[];
  refreshing: boolean;
  onRefresh: () => void;
  onEndReached: () => void;
  footer: string | null;
}): React.JSX.Element;
```

QUÉ DECIR MIENTRAS LO ESCRIBES

El orden en el que lo escribo

Empiezo por el bug, no por los tipos. "Si vacío el array al refrescar, un error me deja la pantalla en blanco. Así que el estado bueno se queda hasta que la página nueva llega."

Después la generación. Subo el número antes del await y, al volver, comparo. Cuento el caso: refresh en vuelo, la página anterior resuelve después, su `requestId` ya no coincide, se tira. El `AbortController` es la optimización para no gastar red. La generación es la corrección, porque el fetcher puede ignorar la señal.

El `return` de `loadMore` sin cursor va antes de tocar `generation`. Si lo pongo después, un `onEndReached` de una lista corta aborta el refresh que sí importaba. Lo digo, porque es el bug que yo mismo metería con prisa.

`fetchRef` evita que una lambda nueva rearme `run`. El effect de la pantalla depende de `refresh`, y `refresh` es estable.

En la pantalla dejo los cuatro estados visibles: cargando por primera vez, error sin datos con reintento, vacío de verdad, y lista con banner si el error llegó cuando ya había filas. `hasMore` sale del cursor. No de `items.length % 20`.

Tests que enumero, aunque no los teclee: refresh que convive con un append lento y gana el refresh; `loadMore` con cursor null no incrementa la generación; error de red conserva items; dos páginas con un id repetido no duplican la fila; unmount aborta y un resolve posterior no hace `setState` útil. En React 19 el setState tras unmount no avisa, así que el test afirma el resultado, no la ausencia de un warning.

TRAMPAS

- Abortar y no subir la generación. Si el fetch no mira la señal, el resultado viejo igual entra.
- Poner items: [] en el setState del comienzo del refresh.
- Depender de fetchPage en el useCallback y crear el hook en un render que también crea el fetcher.
- Reintentar el error desde onEndReached. El footer tiene un botón. El callback de la lista no es un scheduler.
- hasMore leído de un ref durante el render sin que un setState lo acompañe. En esta solución el setState del fin de la request vuelve a pintar, y ahí el ref ya tiene el cursor nuevo. Si alguien lee hasMore a mitad de la request, puede ir un frame atrás. Lo digo si me queda aire.

EJERCICIO 03 · 8 MIN

El sheet que persigue al dedo

ENUNCIADO

Este bottom sheet se mueve desde JS. A 120 Hz el dedo va por delante de la vista, y soltar dispara decenas de `setState`.

Tienes unos ocho minutos para pasarlo al runtime de UI. Tres snaps: abierto a 80, medio a 320, cerrado a 560. Resistencia al pasar los extremos. Al soltar, cae al snap más cercano a la proyección de la velocidad. React solo se entera si el destino es el cerrado.

No hace falta estilizar. Hace falta que el worklet no vuelva a JS en `onUpdate`.

LO QUE EL ENTREVISTADOR MIRA

- Gesture.Pan, no useAnimatedGestureHandler ni PanResponder.
- La posición vive en un shared value. El punto de partida se copia en onStart a otro shared value.
- scheduleOnRN(close) una sola vez, en onEnd, y con el API de un solo call.
- La resistencia y el snap son matemáticas en el worklet, no setState.
- Mencionas que el plugin de worklets va último y que el root es GestureHandlerRootView.

CÓDIGO INICIAL

```
import { useState } from "react";
import { PanResponder, View } from "react-native";
import Animated, { useSharedValue } from "react-native-reanimated";

const OPEN = 80;
const CLOSED = 560;

export function Sheet({ onClose }: { onClose: () => void }) {
  const [top, setTop] = useState(CLOSED);
  const translateY = useSharedValue(CLOSED);

  const pan = PanResponder.create({
    onMoveShouldSetPanResponder: () => true,
    onPanResponderMove: (_evt, gesture) => {
      const next = CLOSED + gesture.dy;
      translateY.value = next;
      setTop(next);
      if (next > CLOSED - 40) onClose();
    },
  });

  return (
    <Animated.View
      {...pan.panHandlers}
      style={{ transform: [{ translateY: top }] }}
    />
  );
}
```

DIAGNÓSTICO, ANTES DE REESCRIBIR

Por qué va tarde

- `PanResponder` entrega el movimiento al hilo de JS.
- `setTop` en cada evento programa un render de React para mover una vista. A 120 Hz ese render no cabe en 8 ms en cuanto hay algo más en la pantalla.

- El shared value se escribe y no se lee. El estilo usa `top`, el estado de React. La vista espera al commit.
- `onClose` durante el move desmonta el sheet mientras el dedo sigue abajo. Tiene que decidirse al soltar, y solo si el snap elegido es el cerrado.
- `useAnimatedGestureHandler` no es la migración: en Reanimated 4 no está.

El estilo animado tiene que leer el shared value. El `transform` se anima. El `top` no.

SOLUCIÓN

```
import { useCallback, useRef } from "react";
import { StyleSheet } from "react-native";
import { Gesture, GestureDetector } from "react-native-gesture-handler";
import Animated, {
  useAnimatedStyle,
  useSharedValue,
  withSpring,
} from "react-native-reanimated";
import { scheduleOnRN } from "react-native-worklets";

const OPEN = 80;
const MID = 320;
const CLOSED = 560;
const SNAPS = [OPEN, MID, CLOSED];

export function Sheet({ onClose }: { onClose: () => void }) {
  const translateY = useSharedValue(CLOSED);
  const startY = useSharedValue(CLOSED);
  const onCloseRef = useRef(onClose);
  onCloseRef.current = onClose;

  const close = useCallback(() => {
    onCloseRef.current();
  }, []);

  const pan = Gesture.Pan()
    .onStart(() => {
      startY.value = translateY.value;
    })
    .onUpdate((event) => {
      const next = startY.value + event.translationY;
      if (next < OPEN) {
        translateY.value = OPEN + (next - OPEN) * 0.28;
      } else if (next > CLOSED) {
        translateY.value = CLOSED + (next - CLOSED) * 0.28;
      } else {
        translateY.value = next;
      }
    })
    .onEnd((event) => {
      const projected = translateY.value + event.velocityY * 0.18;
      let dest = SNAPS[0];
      let best = Math.abs(projected - dest);
      for (let index = 1; index < SNAPS.length; index += 1) {
        const distance = Math.abs(projected - SNAPS[index]);
        if (distance < best) {
          best = distance;
          dest = SNAPS[index];
        }
      }
      translateY.value = withSpring(dest, { velocity: event.velocityY });
      if (dest === CLOSED) scheduleOnRN(close);
    });

  const animatedStyle = useAnimatedStyle(() => ({
```

```
    transform: [{ translateY: translateY.value }],
  }));

  return (
    <GestureDetector gesture={pan}>
      <Animated.View style={[styles.sheet, animatedStyle]} />
    </GestureDetector>
  );
}

const styles = StyleSheet.create({
  sheet: {
    position: "absolute",
    left: 0,
    right: 0,
    height: 640,
    backgroundColor: "#f4efe2",
  },
});
```

QUÉ DECIR MIENTRAS LO ESCRIBES

Narración

"El dedo se queda en el UI thread. Por eso dejo PanResponder." Escribo `Gesture.Pan`. Sus callbacks ya son worklets. No pongo `runOnJS(true)`.

`startY` es un shared value, no un `let` del render. Si fuera un `let`, el worklet se quedaría con el número del último render de React, que durante el gesto no existe, porque React no está pintando.

En `onUpdate` solo escribo `translateY`. Cuento la resistencia: fuera de rango, el dedo recorre 1 y la vista recorre 0,28. Al soltar, el spring la devuelve. Si en el extremo escribiera `clamp` duro, el dedo se despega igual, solo que contra una pared.

En `onEnd` proyecto con la velocidad. El coeficiente 0,18 es un punto de partida, no una ley física: lo afinaría con el dedo en un teléfono de 120 Hz. Elijo el snap por distancia a esa proyección, no a la posición cruda. Sin velocidad, un flick corto no cambia de snap y el sheet se siente pegado.

`scheduleOnRN(close)` va después de decidir, y solo si cerró. El API es de un argumento extra, no `runOnJS(close)()`. `close` es estable y lee un ref, porque `onClose` del padre puede cambiar de identidad sin que yo quiera recrear la decisión.

El estilo es `transform`. Digo la frase: animar `top` obliga a layout, animar `translateY` no.

Si me sobra un minuto: `GestureHandlerRootView` una sola vez en la raíz, el plugin `react-native-worklets/plugin` al final de Babel, y el import del gesture handler en el entry. Y `withSpring` en la 4 cambió el reposo a `energyThreshold`. No porto un duration de la 3 esperando el mismo tiempo, porque el duration nuevo es perceptual.

TRAMPAS

- Llamar scheduleOnRN dentro de onUpdate para 'ir actualizando el estado'. Volviste a JS cada frame.
- Capturar start en una variable de JS en vez de un shared value.
- onClose directo dentro del worklet. Puede funcionar de casualidad y romperse cuando la función deje de ser shareable. El camino soportado es scheduleOnRN.
- Montar el sheet sin GestureHandlerRootView y concluir que Reanimated está roto.
- Sumar el alto del teclado con setState. Si sale el tema, el teclado es otro shared value, desde keyboard-controller.

DIEZ MINUTOS

Ficha de repaso

Léela en la antesala. No reemplaza la hora: fija versiones, nombres que cambiaron y las frases que delatan un blog de 2023.

Versiones que se dicen solas

- React Native 0.88 es el estable desde el 15 de octubre de 2026. React 19.3. Expo SDK 58. 0.87 sigue en soporte activo. 0.89 todavía no salió.
- New Architecture obligatoria desde 0.82 (octubre 2025). newArchEnabled=false se ignora.
- 0.84 (febrero 2026): Hermes V1 por defecto, legacy compilado fuera en iOS, binarios precompilados del core.
- 0.88: eventos por el EventTarget de la web, SceneDelegate en iOS, sin breaking changes de producto respecto de 0.87.
- SDK 58 quitó InteractionManager (usa requestIdleCallback), el export raíz Touchable, y varias props de StatusBar: backgroundColor, translucent, networkActivityIndicatorVisible. ImageBackground quedó deprecado.
- API de TypeScript estricta por defecto desde 0.87. Deep imports de react-native/Libraries son error. Los refs son tipos de instancia.
- El remote debugging de Chrome murió en 0.79. Se depura con React Native DevTools.

Relojes

- 16,6 ms por frame a 60 Hz. 8,3 ms a 120 Hz.
- JS fps bajo y UI fps alto: Hermes. Parse, sort, render, TurboModule síncrono, console.log enorme.
- UI fps bajo y JS fps alto: main thread. Vistas de más, imagen sin tamaño, sombra, blur, animar width o top.
- El runtime de Reanimated es otro JS, en el UI thread. No comparte el heap con React.
- scheduleOnRN vuelve a React. Una vez por gesto, no por frame.

Nombres que cambiaron

- runOnJS(fn)(arg) pasó a scheduleOnRN(fn, arg), desde react-native-worklets.
- runOnUI pasó a scheduleOnUI. executeOnUIRuntimeSync pasó a runOnUISync.
- El plugin de Babel es react-native-worklets/plugin y va el último. React Compiler, si lo agregas a mano, va el primero.
- useAnimatedGestureHandler se eliminó en Reanimated 4. El gesto es Gesture.Pan().
- useScrollViewOffset se llama useScrollOffset.
- cacheTime de TanStack Query v5 se llama gcTime.
- Reanimated 4 no corre en la arquitectura vieja ni sobre el motor de V8. La 3 no es un plan de downgrade.
- FlashList v2 no te pide estimatedItemSize.
- eject no existe. El camino es prebuild, config plugins y, si hace falta, commitear las carpetas.

Listas en una frase cada una

- FlatList desmonta. FlashList v2 recicla. El estado local de una celda reciclada se resetea con el id.
- keyExtractor devuelve id. El índice miente en cuanto la lista se filtra o se le inserta algo arriba.
- getItemLayout, en FlatList, es lo que vuelve de fiar a scrollToIndex. En FlashList v2 el scroll refina midiendo.
- onEndReached recuerda el cursor que ya pidió. Un error no se reintenta solo desde ese callback.
- expo-image con ancho, alto y recyclingKey. Una foto de cámara en un avatar de 36 pt es jank de UI.
- No hay VirtualizedList dentro de un ScrollView. Horizontal dentro de vertical, con alto explícito, sí.

Estado

- Query es el servidor. Zustand es la UI. Context es una dependencia estable. El saldo no se copia en un useEffect.
- staleTime 0 significa siempre vencido. Infinity significa que solo cambia si invalidas.
- MMKV para la caché y la bandeja, cifrado, con la clave en SecureStore. AsyncStorage no hidrata el primer frame.
- El token vive en el llavero. Nunca en el bundle, nunca en la query persistida en claro.
- Un pago no es optimista. Lleva llave de idempotencia y se muestra pendiente hasta el ack.
- structuralSharing conserva la referencia si el JSON no cambió. Un map que clona despierta a todas las filas.

Release

- El binario fija Hermes. El OTA solo lleva JS que ese Hermes ejecuta. Bytecode de otra versión mata el arranque: Wrong bytecode version.
- Entre 0.82 y 0.83 el bytecode era 96. Entre 0.84 y 0.86 fue 98. En 0.88 no recites el entero: recita el contrato.
- runtimeVersion con fingerprint, o un string que cambia en el mismo PR que el nativo.
- Updates firmados. La pública va en el binario. La privada, en CI.
- Source maps del binario y source maps de cada update. Son artefactos distintos.
- FLAG_SECURE en Android para pantallas de dinero. En iOS, blur en el app switcher. Eso no es anti-jailbreak.
- Pinning solo con runbook de rotación. Si no existe el runbook, TLS es la respuesta honesta.

Frases que te bajan de senior

- Activemos el bridge nuevo.
- Ponemos newArchEnabled en false mientras estabilizamos.
- Lo miro en Flipper. O con el remote debugger de Chrome.
- Después del gesto uso InteractionManager.
- FlashList siempre necesita estimatedItemSize.
- Lo pruebo en Expo Go, aunque el módulo sea nativo nuestro.
- Hay que hacer eject.
- El secreto está seguro porque Hermes lo compiló.
- memo en todos los componentes y llegamos a 60 fps.
- Detox para los cuatrocientos flujos.

CIERRE

Lo que el binario fija

El motor vive en el binario. El update solo puede traer JavaScript que ese motor sabe ejecutar. El secreto no viaja en ninguno de los dos artefactos. Un crash sin símbolo no es un crash: es un rumor.